

TU ES UN EXPERT EN UI UX DESIGN ET DEVELOPEMENT WEB AVEC PLUS DE 15 ANS D EXPERIENCES ET TU VA SGENERER LE CAHIER DE CHARGES DE KHALILSHOP UN SITE E COMMERCE ULTRA MODERNE AVEC FRONTEND TAILWIND CSS BACKEND LARAVEL

 Modifier



CAHIER DES CHARGES — KHALILSHOP

Site e-commerce ultra-moderne de vente de produits Apple

Version : 1.0

Projet : KhalilShop

Type : Plateforme e-commerce B2C

Frontend : Tailwind CSS + Blade + Alpine.js

Backend : Laravel

Base de données : PostgreSQL ou MySQL

Stockage images : Cloudinary

Paiement : À intégrer selon les moyens de paiement retenus

Design : Premium, minimaliste, moderne et orienté conversion

1. Présentation du projet

KhalilShop est une plateforme e-commerce moderne spécialisée dans la vente de produits Apple : iPhone, iPad, MacBook, Apple Watch, AirPods et accessoires.

L'objectif est de créer une expérience utilisateur comparable aux meilleurs sites e-commerce modernes, tout en l'adaptant au marché sénégalais.

Le site devra être :

- rapide ;
- responsive ;
- sécurisé ;
- intuitif ;
- élégant ;
- optimisé pour mobile ;
- optimisé pour le référencement naturel ;
- facilement administrable ;
- évolutif ;
- orienté conversion.

L'identité visuelle devra privilégier une approche **Apple Premium / Luxury Tech**, avec beaucoup d'espace, des animations discrètes, des images produits de haute qualité et une navigation fluide.

2. Objectifs du projet

2.1 Objectif principal

Créer une plateforme permettant aux clients de :

- consulter les produits ;
- rechercher rapidement un produit ;
- filtrer les produits ;
- consulter les détails d'un produit ;
- choisir une couleur ;
- choisir une capacité de stockage ;
- sélectionner une variante ;
- ajouter un produit au panier ;
- modifier le panier ;
- passer une commande ;
- choisir un mode de livraison ;
- choisir un moyen de paiement ;
- suivre l'état de leur commande ;
- créer et gérer leur compte.

2.2 Objectifs commerciaux

Le site devra également permettre à KhalilShop de :

- augmenter les ventes ;
 - mettre en avant les promotions ;
 - mettre en avant les nouveautés ;
 - gérer facilement les stocks ;
 - gérer les produits ;
 - gérer les commandes ;
 - gérer les clients ;
 - gérer les promotions ;
 - analyser les performances commerciales.
-

3. Public cible

La plateforme cible principalement :

Particuliers

Clients souhaitant acheter :

- iPhone ;
- iPad ;
- MacBook ;
- Apple Watch ;
- AirPods ;
- accessoires Apple.

Professionnels

Entreprises recherchant :

- ordinateurs ;
- smartphones ;
- équipements informatiques ;
- solutions d'équipement pour leurs employés.

Clients recherchant des facilités de paiement

Le site pourra proposer un système de paiement échelonné, notamment pour les produits éligibles à une offre de financement

telle que Lebalma.

4. Identité visuelle

4.1 Palette principale

La direction artistique reposera sur une identité premium.

Couleurs principales

Noir profond

#111827

Utilisation :

- navbar ;
- footer ;
- boutons importants ;
- sections premium ;
- éléments de contraste.

Blanc

#FFFFFF

Utilisation :

- arrière-plans ;
- cartes produits ;
- textes sur fonds sombres ;
- espaces de respiration.

Bleu électrique

#0A84FF

Utilisation :

- boutons ;
- liens ;
- badges ;
- éléments interactifs ;
- promotions ;

- états actifs.

4.2 Typographie

Police principale :

Montserrat

Utilisation :

- titres ;
- navigation ;
- descriptions ;
- boutons ;
- prix ;
- éléments marketing.

La hiérarchie typographique devra être très claire :

- H1 : très grand et impactant ;
 - H2 : section ;
 - H3 : sous-section ;
 - texte courant : lisible ;
 - prix : très visible ;
 - informations secondaires : discrètes.
-

5. Principes UX/UI

Le design devra respecter les principes suivants :

Simplicité

Le client doit pouvoir trouver un produit en quelques secondes.

Hiérarchie visuelle

Les éléments les plus importants doivent être immédiatement visibles :

1. produit ;
2. prix ;
3. promotion ;
4. disponibilité ;

5. appel à l'action.

Mobile First

Le site devra être pensé en priorité pour smartphone.

Micro-interactions

Ajouter des animations légères :

- hover ;
- apparition des cartes ;
- transition des boutons ;
- changement d'image ;
- ouverture du panier ;
- filtres ;
- notifications.

Les animations devront rester rapides et professionnelles.

Aucun effet excessif

Le site ne devra pas être surchargé d'animations.

L'objectif est de donner une impression :

Premium + rapide + moderne + technologique.

6. Architecture générale du site

Le site devra comporter les principales pages suivantes :

- Accueil ;
- Boutique ;
- Catégories ;
- Produit ;
- Recherche ;
- Promotions ;
- Nouveautés ;
- Panier ;
- Checkout ;
- Connexion ;
- Inscription ;

- Mon compte ;
 - Mes commandes ;
 - Suivi de commande ;
 - À propos ;
 - Contact ;
 - FAQ ;
 - Conditions générales ;
 - Politique de confidentialité.
-

7. Header / Navigation

Le header devra être particulièrement travaillé car il constitue l'un des éléments principaux de navigation.

7.1 Première barre

Sur desktop :

Logo | Recherche | Compte | Panier

La barre de recherche devra occuper une grande partie de l'espace disponible.

Elle permettra de rechercher :

- iPhone 15 ;
- iPhone 17 Pro Max ;
- MacBook Air ;
- iPad Air ;
- AirPods ;
- etc.

7.2 Deuxième barre

Une barre de catégories permettra d'accéder rapidement à :

- iPhone ;
- iPad ;
- MacBook ;
- Apple Watch ;
- AirPods ;
- Accessoires ;

- Promotions ;
- Nouveautés.

7.3 Mobile

Sur mobile :

- logo ;
- recherche ;
- panier ;
- menu hamburger.

Le menu devra s'ouvrir sous forme de panneau ou drawer.

8. Mega Menu

Le menu desktop pourra utiliser un mega menu moderne.

Exemple :

iPhone

- iPhone 17
- iPhone 17 Air
- iPhone 17 Pro
- iPhone 17 Pro Max
- iPhone 16
- iPhone 15
- iPhone 14
- iPhone 13
- iPhone 12
- iPhone XR

iPad

- iPad
- iPad Air
- iPad Pro
- Accessoires iPad

Mac

- MacBook Air

- MacBook Pro
- Mac mini
- Accessoires Mac

Accessoires

- AirPods
 - Chargeurs
 - Câbles
 - Coques
 - Adaptateurs
-

9. Page d'accueil

La page d'accueil devra être conçue comme une véritable page commerciale.

9.1 Hero Section

Grand visuel avec :

- produit phare ;
- titre marketing ;
- description courte ;
- prix ;
- bouton "Acheter maintenant".

Exemple :

Découvrez la nouvelle génération d'iPhone

"Performance. Design. Intelligence."

Bouton :

Découvrir

Deuxième bouton :

Voir les produits

10. Bannière paiement échelonné

Une section dédiée pourra présenter l'offre de paiement en plusieurs fois.

Exemple :

Votre iPhone maintenant. Payez progressivement.

Informations :

- acompte ;
- durée ;
- produits éligibles ;
- conditions.

Bouton :

Découvrir l'offre

Cette section devra être visuellement différente du reste de la page.

11. Section Nouveautés

Afficher les derniers produits ajoutés.

Chaque carte produit devra afficher :

- image ;
 - badge "Nouveau" ;
 - nom ;
 - modèle ;
 - prix ;
 - ancien prix si promotion ;
 - réduction ;
 - disponibilité ;
 - bouton panier.
-

12. Section Promotions

Les produits en promotion devront être regroupés dans une section dédiée.

Exemple :

Offres exceptionnelles

Carte :

iPhone 17 Pro Max

Prix avant :

~~1 125 000 FCFA~~

Prix actuel :

900 000 FCFA

Badge :

-20%

13. Catégories

Une section visuelle permettra de présenter les catégories.

Exemple :

- iPhone ;
- iPad ;
- MacBook ;
- Apple Watch ;
- AirPods ;
- Accessoires.

Chaque catégorie pourra utiliser une grande image avec un effet hover.

14. Catalogue / Boutique

La page boutique sera l'une des pages principales.

Elle devra proposer :

Barre supérieure

- nombre de produits ;
- tri ;
- affichage grille/liste.

Filtres

Filtrer par :

- catégorie ;
- marque ;
- prix ;
- disponibilité ;
- modèle ;
- couleur ;
- stockage ;
- promotion ;
- nouveauté.

Tri

- Pertinence ;
 - Prix croissant ;
 - Prix décroissant ;
 - Plus récent ;
 - Meilleures ventes.
-

15. Carte produit

La carte produit devra être minimaliste et premium.

Structure :

Image

Badge éventuel :

NOUVEAU

ou

PROMO

Puis :

iPhone 17 Pro Max

256 Go

Prix :

900 000 FCFA

Ancien prix :

~~1 125 000 FCFA~~

Actions :

- Ajouter au panier ;
 - Voir le produit ;
 - Ajouter aux favoris.
-

16. Page détail produit

La page produit devra être très complète.

Galerie

Afficher :

- image principale ;
- miniatures ;
- plusieurs angles ;
- images de variantes.

Possibilité de zoom.

Informations

Afficher :

- nom ;
- modèle ;
- disponibilité ;
- prix ;
- ancien prix ;
- réduction ;
- description ;

- caractéristiques.

Variantes

Le client pourra sélectionner :

Couleur

Exemple :

- Noir ;
- Blanc ;
- Bleu ;
- Titane naturel.

Stockage

Exemple :

- 128 Go ;
- 256 Go ;
- 512 Go ;
- 1 To.

Le prix pourra changer automatiquement selon la variante.

17. Quantité

Le client pourra sélectionner :

- 1 +

Puis :

Ajouter au panier

18. Informations de confiance

Sous le bouton d'achat :

- Produit garanti ;
- Livraison partout au Sénégal ;

- Paiement sécurisé ;
- Service client ;
- Produit authentique.

Ces éléments devront rassurer le client avant l'achat.

19. Produits similaires

En bas de la page produit :

Vous pourriez également aimer

Afficher :

- produits similaires ;
 - produits de la même catégorie ;
 - accessoires compatibles.
-

20. Panier

Le panier pourra fonctionner comme un drawer sur desktop et mobile.

Afficher :

- produit ;
- image ;
- variante ;
- quantité ;
- prix ;
- suppression.

Résumé :

Sous-total

Livraison

Total

Bouton :

21. Checkout

Le checkout devra être simple et rapide.

Étapes :

1. Informations client

- prénom ;
- nom ;
- téléphone ;
- email.

2. Adresse

- région ;
- ville ;
- quartier ;
- adresse ;
- indication supplémentaire.

3. Livraison

Options configurables depuis l'administration.

4. Paiement

Moyens de paiement à intégrer selon les partenaires retenus.

5. Confirmation

Résumé complet :

- produits ;
- quantités ;
- livraison ;
- paiement ;
- total.

Bouton :

Confirmer la commande

22. Compte client

Le client pourra créer un compte.

Fonctionnalités :

- inscription ;
 - connexion ;
 - récupération du mot de passe ;
 - modification du profil ;
 - changement de mot de passe ;
 - gestion des adresses ;
 - historique des commandes ;
 - suivi des commandes ;
 - favoris.
-

23. Suivi des commandes

Le client pourra consulter le statut de sa commande.

Statuts :

Commande reçue



Commande confirmée



En préparation



Expédiée



En livraison



Livrée

L'interface pourra présenter ces étapes sous forme de timeline.

24. Système de favoris

Le client pourra ajouter un produit à ses favoris.

Fonctionnalités :

- ajouter ;
 - supprimer ;
 - consulter ;
 - ajouter directement au panier.
-

25. Recherche avancée

La recherche devra être dynamique.

Lors de la saisie :

iphone 17

le système pourra afficher :

- produits correspondants ;
- catégories ;
- suggestions ;
- résultats populaires.

Exemple :

Rechercher "iphone"

- iPhone 17
 - iPhone 17 Pro
 - iPhone 17 Pro Max
 - iPhone 16
 - iPhone 15
-

26. Notifications

Le site devra utiliser des notifications modernes de type toast.

Exemples :

Produit ajouté au panier

Commande confirmée

Mot de passe modifié

Produit indisponible

Les notifications devront être discrètes et élégantes.

27. Footer

Le footer devra comporter plusieurs colonnes.

KhalilShop

- À propos ;
- Contact ;
- Notre boutique.

Produits

- iPhone ;
- iPad ;
- MacBook ;
- Apple Watch ;
- AirPods.

Assistance

- FAQ ;
- Livraison ;
- Retours ;
- Garantie ;
- Contact.

Informations

- Conditions générales ;
- Politique de confidentialité ;

- Conditions de vente.

Réseaux sociaux

- Instagram ;
 - Facebook ;
 - TikTok ;
 - WhatsApp.
-

28. Back-office administrateur

L'administration constituera une partie majeure du projet.

Elle sera développée avec Laravel.

Dashboard

Afficher :

- chiffre d'affaires ;
- commandes ;
- clients ;
- produits ;
- produits en rupture ;
- commandes en attente ;
- ventes récentes.

Des graphiques permettront de visualiser :

- ventes par jour ;
 - ventes par mois ;
 - produits les plus vendus ;
 - chiffre d'affaires ;
 - évolution des commandes.
-

29. Gestion des produits

L'administrateur pourra :

- créer un produit ;
- modifier un produit ;

- supprimer un produit ;
 - désactiver un produit ;
 - gérer le stock ;
 - gérer le prix ;
 - gérer l'ancien prix ;
 - gérer les promotions ;
 - gérer les variantes ;
 - gérer les couleurs ;
 - gérer les stockages ;
 - gérer les images.
-

30. Gestion des images

Les images produits seront stockées sur Cloudinary.

Le système devra permettre :

- upload ;
- compression ;
- optimisation ;
- génération de plusieurs formats ;
- suppression ;
- remplacement.

Les URLs Cloudinary seront enregistrées dans la base de données.

31. Gestion des catégories

L'administrateur pourra :

- créer une catégorie ;
 - modifier ;
 - supprimer ;
 - activer/désactiver ;
 - ajouter une image ;
 - modifier l'ordre d'affichage.
-

32. Gestion des commandes

L'administrateur pourra consulter :

- numéro de commande ;
- client ;
- téléphone ;
- adresse ;
- produits ;
- quantité ;
- montant ;
- livraison ;
- paiement ;
- statut.

Actions :

- confirmer ;
 - préparer ;
 - expédier ;
 - livrer ;
 - annuler.
-

33. Gestion des clients

L'administrateur pourra consulter :

- nom ;
 - email ;
 - téléphone ;
 - commandes ;
 - montant dépensé ;
 - date d'inscription.
-

34. Gestion des promotions

L'administrateur pourra créer :

- réduction en pourcentage ;
- réduction fixe ;
- code promotionnel ;
- date de début ;

- date de fin ;
- produits concernés.

Exemple :

PROMO20

-20%

35. Gestion des bannières

L'administrateur pourra gérer les éléments de la page d'accueil :

- hero ;
- promotions ;
- nouveautés ;
- campagnes ;
- bannières.

Cela permettra de modifier le contenu commercial sans modifier le code.

36. Architecture technique Laravel

Le projet devra suivre une architecture propre et maintenable.

Structure recommandée :

```
Laravel
|
├─ app/
|   ├─ Models/
|   ├─ Http/
|   │   ├─ Controllers/
|   │   ├─ Requests/
|   │   └─ Middleware/
|   └─ Services/
|       └─ Policies/
|
├─ database/
|   ├─ migrations/
|   └─ seeders/
```

```
|   └─ factories/
|
|─ resources/
|   └─ views/
|       └─ css/
|           └─ js/
|
|─ routes/
|   └─ web.php
|       └─ api.php
|
|─ storage/
|
└─ tests/
```

37. Stack technique

Frontend

- HTML5 ;
- Tailwind CSS ;
- Blade ;
- Alpine.js ;
- JavaScript ;
- Vite.

Backend

- Laravel ;
- PHP ;
- Laravel Eloquent ORM ;
- Laravel Validation ;
- Laravel Authentication ;
- Laravel Policies ;
- Laravel Notifications.

Base de données

- PostgreSQL ou MySQL.

Images

- Cloudinary.

Versioning

- Git ;
 - GitHub.
-

38. Eloquent ORM

Les relations Laravel Eloquent devront être utilisées pour gérer les données.

Relations principales :

Category

```
|
└─ hasMany Products
```

Product

```
|
├─ belongsTo Category
├─ hasMany ProductVariants
└─ hasMany OrderItems
```

User

```
|
├─ hasMany Orders
└─ hasMany Favorites
```

Order

```
|
├─ belongsTo User
└─ hasMany OrderItems
```

OrderItem

```
|
└─ belongsTo Product
```

39. Modèle de données principal

Tables principales :

users

- id ;
- name ;
- email ;
- phone ;
- password ;
- role ;
- timestamps.

categories

- id ;
- name ;
- slug ;
- description ;
- image ;
- is_active ;
- timestamps.

products

- id ;
- category_id ;
- name ;
- slug ;
- description ;
- model ;
- price ;
- old_price ;
- stock ;
- is_new ;
- is_promo ;
- is_featured ;
- timestamps.

product_variants

- id ;
- product_id ;
- color ;
- storage ;

- price ;
- stock ;
- sku.

product_images

- id ;
- product_id ;
- url ;
- public_id ;
- alt ;
- sort_order.

orders

- id ;
- user_id ;
- order_number ;
- status ;
- subtotal ;
- shipping_cost ;
- total ;
- payment_status ;
- payment_method ;
- shipping_address ;
- timestamps.

order_items

- id ;
- order_id ;
- product_id ;
- variant_id ;
- quantity ;
- unit_price ;
- total.

favorites

- id ;
- user_id ;
- product_id.

coupons

- id ;
 - code ;
 - type ;
 - value ;
 - starts_at ;
 - expires_at ;
 - is_active.
-

40. Sécurité

Le système devra respecter les bonnes pratiques Laravel.

Mesures :

- authentification sécurisée ;
- hashage des mots de passe ;
- CSRF ;
- validation serveur ;
- protection XSS ;
- protection SQL Injection via Eloquent ;
- autorisations avec Policies/Gates ;
- limitation des requêtes sensibles ;
- validation des uploads ;
- contrôle des rôles ;
- HTTPS en production.

Les routes d'administration devront être protégées.

41. Gestion des rôles

Au minimum :

Client

Accès :

- catalogue ;
- panier ;

- commandes ;
- compte ;
- favoris.

Administrateur

Accès :

- dashboard ;
- produits ;
- catégories ;
- commandes ;
- clients ;
- promotions ;
- contenu ;
- paramètres.

Une évolution pourra ajouter :

Gestionnaire

Accès limité à :

- produits ;
- stock ;
- commandes.

42. Responsive Design

Le site devra être parfaitement adapté :

- smartphone ;
- tablette ;
- ordinateur portable ;
- desktop ;
- grands écrans.

Breakpoints Tailwind :

sm

md

lg

x1

2x1

Aucun élément ne devra provoquer de scroll horizontal involontaire.

43. Performance

Le site devra être optimisé pour obtenir de bonnes performances.

Mesures :

- images optimisées ;
- lazy loading ;
- responsive images ;
- cache Laravel ;
- pagination ;
- requêtes Eloquent optimisées ;
- eager loading ;
- minification ;
- Vite ;
- réduction du JavaScript ;
- CDN pour les images.

Objectif :

chargement rapide même avec une connexion mobile.

44. SEO

Chaque produit devra disposer de :

- URL propre ;
- slug ;
- title ;
- meta description ;
- image Open Graph ;
- données structurées lorsque pertinent.

Exemple :

/khalilshop/iphone-17-pro-max

et non :

/product?id=152

45. Accessibilité

Le site devra respecter les bonnes pratiques d'accessibilité :

- contraste suffisant ;
 - textes alternatifs ;
 - navigation clavier ;
 - boutons clairement identifiables ;
 - labels de formulaires ;
 - structure HTML sémantique ;
 - messages d'erreur compréhensibles.
-

46. Analytics

Le système pourra intégrer :

- Google Analytics ;
- Google Search Console ;
- Meta Pixel.

Événements importants :

- consultation produit ;
 - ajout au panier ;
 - début checkout ;
 - achat ;
 - recherche ;
 - inscription.
-

47. Architecture des URLs

Exemple :

/

/shop

```
/category/iphone
/category/ipad
/category/macbook
/product/iphone-17-pro-max
/promotions
/new
/cart
/checkout
/login
/register
/account
/account/orders
/account/orders/{order}
/contact
/faq
```

Administration :

```
/admin
/admin/dashboard
/admin/products
/admin/products/create
/admin/categories
/admin/orders
/admin/customers
/admin/promotions
/admin/banners
/admin/settings
```

48. API

Même si Laravel utilise Blade comme frontend principal, l'architecture devra être pensée de façon à pouvoir exposer ultérieurement une API.

Exemples :

```
GET    /api/products
GET    /api/products/{id}
POST   /api/cart
POST   /api/orders
GET    /api/orders
```



```
POST    /api/auth/login
POST    /api/auth/register
```

Cela permettra éventuellement de développer plus tard :

- application mobile ;
- application React ;
- application Flutter ;
- application React Native.

49. Gestion du stock

Le stock devra être automatiquement mis à jour après validation d'une commande.

Exemple :

Stock initial :

```
10
```

Commande :

```
2
```

Nouveau stock :

```
8
```

Le produit devra automatiquement passer en :

Rupture de stock

lorsque :

```
stock = 0 .
```

50. Gestion des variantes

Un produit pourra posséder plusieurs variantes.

Exemple :

iPhone 17 Pro Max

Couleur	Stockage	Prix
Noir	256 Go	900 000 FCFA
Noir	512 Go	1 000 000 FCFA
Blanc	256 Go	900 000 FCFA
Bleu	512 Go	1 000 000 FCFA

La sélection d'une variante devra actualiser :

- prix ;
- stock ;
- SKU ;
- image correspondante.

51. Livraison

Le système devra permettre de configurer les zones de livraison.

Exemples :

- Dakar ;
- Thiès ;
- Saint-Louis ;
- Mbour ;
- Touba ;
- autres régions.

Le coût pourra dépendre de la zone.

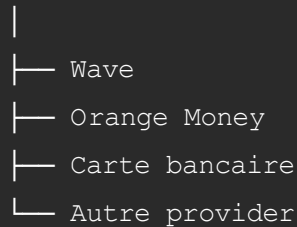
L'administration pourra modifier les tarifs sans intervention du développeur.

52. Paiement

Le système devra être conçu pour pouvoir intégrer plusieurs moyens de paiement.

Architecture recommandée :

PaymentService



Chaque moyen de paiement devra pouvoir être activé/désactivé depuis l'administration lorsque l'intégration le permet.

53. Système de paiement échelonné

Une fonctionnalité spécifique pourra être développée pour les produits éligibles à un paiement en plusieurs fois.

Le produit pourra posséder :

```
eligible_installment = true
```

L'utilisateur pourra voir :

- prix total ;
- acompte ;
- nombre d'échéances ;
- montant par échéance ;
- conditions.

Cette fonctionnalité devra être clairement séparée du paiement classique.

54. Emails et notifications

Le système pourra envoyer automatiquement :

Client

- création de compte ;
- confirmation de commande ;
- paiement confirmé ;
- commande expédiée ;
- commande livrée ;

- réinitialisation du mot de passe.

Administrateur

- nouvelle commande ;
 - paiement reçu ;
 - stock faible.
-

55. États des commandes

Le système devra utiliser une machine d'état claire :

```
pending
confirmed
processing
shipped
delivered
cancelled
```

Les transitions devront être contrôlées côté serveur.

56. Tests

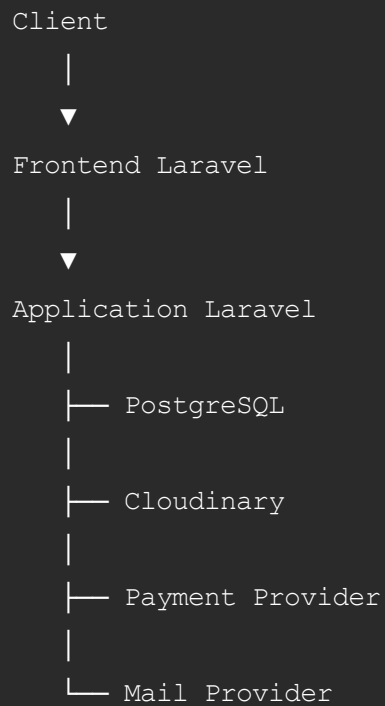
Le projet devra comporter des tests Laravel.

Tests à prévoir :

- authentification ;
 - inscription ;
 - création produit ;
 - modification produit ;
 - panier ;
 - calcul du total ;
 - stock ;
 - commande ;
 - permissions administrateur ;
 - coupons ;
 - paiement.
-

57. Déploiement

Architecture recommandée :



En production :

- HTTPS ;
- variables `.env` ;
- cache Laravel ;
- migrations contrôlées ;
- sauvegardes DB ;
- logs ;
- monitoring.

58. Environnement de développement

Le développement pourra utiliser :

Ubuntu
PHP
Composer
Laravel
Node.js
npm
Tailwind CSS
Vite

PostgreSQL/MySQL

Git

GitHub

Installation et configuration devront être documentées dans un README.

59. Organisation du projet

Le développement sera divisé en phases.

Phase 1 — Analyse

- validation des fonctionnalités ;
- définition des rôles ;
- définition de la base de données ;
- définition du parcours utilisateur.

Phase 2 — UI/UX

- wireframes ;
- maquettes ;
- design system ;
- responsive ;
- validation du parcours.

Phase 3 — Backend Laravel

- installation ;
- base de données ;
- migrations ;
- modèles ;
- relations Eloquent ;
- authentification ;
- API ;
- sécurité.

Phase 4 — Frontend

- layout ;
- navbar ;
- mega menu ;

- homepage ;
- catalogue ;
- produits ;
- panier ;
- checkout ;
- compte.

Phase 5 — Administration

- dashboard ;
- produits ;
- catégories ;
- commandes ;
- clients ;
- promotions ;
- contenu.

Phase 6 — Intégrations

- Cloudinary ;
- paiement ;
- emails ;
- analytics.

Phase 7 — Tests

- tests fonctionnels ;
- tests responsive ;
- tests sécurité ;
- tests performance.

Phase 8 — Production

- serveur ;
- domaine ;
- SSL ;
- base de données ;
- variables d'environnement ;
- déploiement.

60. Design System

Le projet devra disposer d'un mini design system.

Boutons

Primary

Fond :

#0A84FF

Texte :

#FFFFFF

Secondary

Fond :

#111827

Texte :

#FFFFFF

Outline

Fond transparent.

Bordure :

#111827

61. Composants Tailwind

Les composants devront être réutilisables.

Exemples :

Navbar

Topbar

MegaMenu

Button

ProductCard

ProductGrid

ProductGallery

Price

Badge
SearchBar
FilterSidebar
Pagination
Modal
Drawer
Toast
Input
Select
Breadcrumb
Footer

Cela évitera de répéter le même code dans plusieurs pages.

62. Expérience mobile

Sur smartphone, l'utilisateur devra pouvoir :

1. ouvrir le menu ;
2. rechercher un produit ;
3. filtrer ;
4. consulter le produit ;
5. choisir une variante ;
6. ajouter au panier ;
7. passer commande.

Le parcours d'achat devra nécessiter le minimum d'étapes possible.

63. Critères de réussite UX

Le projet sera considéré comme réussi si :

- l'utilisateur comprend immédiatement ce que vend KhalilShop ;
- les produits sont accessibles rapidement ;
- les prix sont clairement visibles ;
- les promotions sont compréhensibles ;
- le panier est facilement accessible ;
- le checkout est simple ;
- le site est parfaitement utilisable sur mobile ;
- le design donne une impression premium ;

- l'administration est simple à utiliser.
-

64. Critères de réussite technique

Le projet devra :

- utiliser Laravel proprement ;
 - utiliser Eloquent ORM ;
 - respecter MVC ;
 - utiliser des migrations ;
 - utiliser des Form Requests pour les validations complexes ;
 - utiliser Policies/Gates pour les permissions ;
 - éviter les requêtes SQL inutiles ;
 - être sécurisé ;
 - être responsive ;
 - être optimisé ;
 - être maintenable.
-

65. Évolutions futures

L'architecture devra permettre d'ajouter ultérieurement :

- application mobile ;
 - programme fidélité ;
 - système de points ;
 - avis clients ;
 - notation des produits ;
 - comparaison de produits ;
 - chat en ligne ;
 - recommandations intelligentes ;
 - gestion avancée des fournisseurs ;
 - gestion des achats ;
 - gestion comptable ;
 - multi-boutiques ;
 - marketplace ;
 - notifications WhatsApp ;
 - application mobile Flutter/React Native.
-

66. Résultat attendu

KhalilShop devra être plus qu'une simple boutique en ligne.

Le résultat attendu est une **plateforme e-commerce premium orientée Apple**, avec :

Design moderne

-

Expérience utilisateur exceptionnelle

-

Catalogue performant

-

Administration complète

-

Paieement et livraison adaptés au Sénégal

-

Architecture Laravel robuste

-

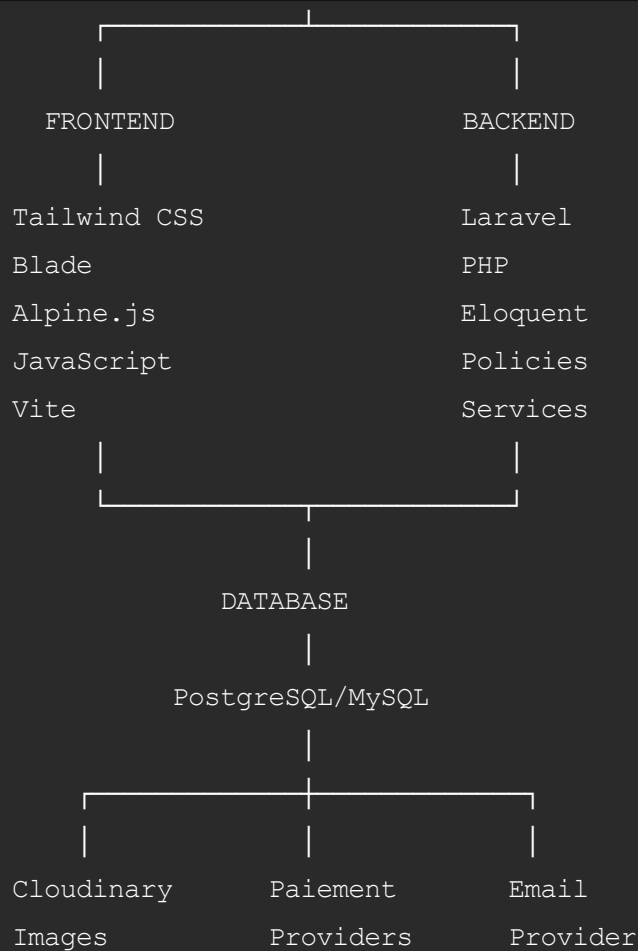
Frontend Tailwind CSS moderne

-

Base technique évolutive

L'objectif final est de donner au client une expérience proche des standards des grandes plateformes e-commerce internationales, tout en conservant une identité propre à KhalilShop et adaptée au marché sénégalais.

67. Stack finale recommandée



Vision finale

**KhalilShop = Premium Apple E-commerce + UX moderne +
Laravel robuste + Tailwind CSS + expérience mobile-first.**

Le développement devra privilégier la **qualité du parcours utilisateur, la performance, la sécurité, la modularité du code et la facilité d'administration**, plutôt que l'accumulation de fonctionnalités inutiles.

NON JE T EN AVAIS PARLER KHALILS SHOP UNE
BOUTIQUE DE VENTE DE VETEMENT HOMME
FEMME CHAUSSURES ACCESOIRE ET VENTE DE
DECO MAISON

Oui, tu as raison. J'avais confondu **KhalilShop** avec ton projet de boutique Apple. Pour **KhalilShop**, le cahier des charges doit concerner une **boutique lifestyle / mode &**

maison, avec :

- 👗 Vêtements femme
- 👕 Vêtements homme
- 👟 Chaussures
- 👛 Accessoires
- 🏠 Décoration maison
- 🛋️ Articles pour la maison

Et la stack reste bien **Frontend Tailwind CSS + Backend Laravel**.

Je peux te refaire le cahier des charges complet de KhalilShop en partant de cette vraie identité, avec notamment **catalogue, variantes taille/couleur, panier, commande, paiement, livraison, favoris, promotions, espace client et back-office**, dans un style **ultra-moderne premium type Zara / H&M / Mango + décoration lifestyle**, mais adapté au marché sénégalais.

OUI

Parfait. Voici la **version corrigée et complète du cahier des charges de KhalilShop**, cette fois entièrement pensée pour une boutique **Mode & Lifestyle** : vêtements homme/femme, chaussures, accessoires et décoration maison.

 Modifier



CAHIER DES CHARGES — KHALILSHOP

Plateforme e-commerce Mode, Accessoires & Décoration Maison

Projet : KhalilShop

Type : Site e-commerce B2C

Positionnement : Mode & Lifestyle Premium

Marché cible : Sénégal et Afrique de l'Ouest

Frontend : Laravel Blade + Tailwind CSS + Alpine.js

Backend : Laravel / PHP

Base de données : PostgreSQL

Images : Cloudinary

Version : 1.0

1. Présentation du projet

KhalilShop est une plateforme e-commerce moderne spécialisée dans la vente de produits de **mode, chaussures, accessoires et décoration maison**.

La boutique doit proposer une expérience digitale premium permettant aux clients de découvrir facilement les collections, consulter les produits, choisir leurs variantes, ajouter des articles au panier et effectuer leurs commandes en ligne.

KhalilShop devra se positionner comme une marque :

moderne, élégante, accessible et tendance.

Le site devra être conçu avec une approche **mobile-first**, car une grande partie des utilisateurs accéderont à la boutique depuis leur smartphone.

L'objectif n'est pas de créer un simple catalogue de produits, mais une véritable **expérience shopping digitale**.

2. Activités de KhalilShop

Le catalogue sera organisé autour de cinq grands univers.

2.1 Femme

- Robes
- Ensembles
- Tops
- Chemisiers
- T-shirts
- Pantalons
- Jeans
- Jupes

- Vestes
- Manteaux
- Pulls
- Tenues tendance

2.2 Homme

- Chemises
- T-shirts
- Polos
- Pantalons
- Jeans
- Shorts
- Ensembles
- Vestes
- Pulls
- Sweats
- Tenues casual
- Tenues élégantes

2.3 Chaussures

- Sneakers
- Baskets
- Sandales
- Escarpins
- Mocassins
- Bottes
- Chaussures homme
- Chaussures femme
- Chaussures casual
- Chaussures élégantes

2.4 Accessoires

- Sacs
- Sacs à main
- Sacs à dos
- Portefeuilles
- Ceintures
- Lunettes
- Bijoux

- Montres
- Casquettes
- Chapeaux
- Foulards

2.5 Maison & Décoration

- Décoration murale
 - Vases
 - Miroirs
 - Bougies
 - Luminaires
 - Coussins
 - Tapis
 - Objets décoratifs
 - Rangement
 - Accessoires de table
 - Décoration chambre
 - Décoration salon
-

3. Objectifs

Objectif principal

Créer une plateforme permettant à KhalilShop de vendre ses produits en ligne de manière professionnelle.

Objectifs secondaires

Le site devra permettre de :

- présenter les collections ;
 - augmenter les ventes ;
 - améliorer la visibilité de la marque ;
 - faciliter les commandes ;
 - gérer les stocks ;
 - gérer les promotions ;
 - fidéliser les clients ;
 - automatiser une partie du processus de vente ;
 - centraliser les commandes.
-

4. Positionnement UX/UI

KhalilShop devra avoir une identité visuelle **fashion + lifestyle + premium**.

L'expérience devra s'inspirer des grands standards des sites de mode modernes :

- grandes photos ;
- espaces blancs généreux ;
- typographie élégante ;
- navigation minimaliste ;
- cartes produits propres ;
- animations discrètes ;
- filtres intuitifs ;
- recherche rapide ;
- checkout simple.

Le site devra éviter l'apparence d'un ancien site e-commerce surchargé.

5. Direction artistique

5.1 Style visuel

Le design devra être :

- minimaliste ;
- élégant ;
- moderne ;
- féminin et masculin à la fois ;
- premium ;
- chaleureux pour la partie décoration ;
- très visuel.

Les photos produits devront occuper une place importante.

6. Palette de couleurs

Une palette moderne pourra être construite autour de :

Noir profond

#111111

Utilisation :

- navigation ;
- titres ;
- boutons ;
- footer.

Blanc cassé

#FAFAF8

Utilisation :

- arrière-plans ;
- sections ;
- cartes.

Beige / Nude

#E8DED2

Utilisation :

- sections lifestyle ;
- décoration ;
- éléments secondaires.

Accent

#C9A27E

Utilisation :

- détails premium ;
- badges ;
- éléments décoratifs.

La palette pourra être ajustée lors de la phase UI Design.

7. Typographie

La typographie devra créer une distinction entre :

Titres

Police élégante pouvant être une serif moderne.

Interface

Une police sans-serif moderne telle que :

Montserrat, Inter ou équivalent.

L'objectif est d'obtenir une combinaison :

Élégance + lisibilité + modernité.

8. Architecture du site

Le site devra comporter :

Accueil
Boutique
Femme
Homme
Chaussures
Accessoires
Maison & Décoration
Nouveautés
Promotions
Collections
Produit
Panier
Checkout
Connexion
Inscription
Mon compte
Mes commandes
Favoris
À propos
Contact
FAQ

9. Header

Le header sera l'un des éléments les plus importants.

Desktop

Organisation :

LOGO

Recherche

Compte

Favoris

Panier

Une deuxième navigation pourra afficher :

Femme

Homme

Chaussures

Accessoires

Maison

Nouveautés

Promotions

10. Recherche

La barre de recherche devra être très visible.

L'utilisateur pourra rechercher :

- robe noire ;
- jean homme ;
- basket blanche ;
- sac à main ;
- miroir ;
- vase ;
- etc.

La recherche pourra afficher des suggestions pendant la saisie.

Exemple :

robe

Produits

- Robe longue noire

- Robe satin beige
 - Robe portefeuille
-

11. Mega Menu

Le mega menu permettra une navigation rapide.

Femme

- Robes
- Tops
- Pantalons
- Jeans
- Ensembles
- Vestes
- Nouveautés

Homme

- T-shirts
- Chemises
- Jeans
- Pantalons
- Vestes
- Ensembles
- Nouveautés

Chaussures

- Sneakers
- Sandales
- Mocassins
- Escarpins
- Bottes

Accessoires

- Sacs
- Bijoux
- Montres
- Lunettes
- Ceintures

Maison

- Salon
 - Chambre
 - Cuisine
 - Décoration
 - Luminaires
 - Rangement
-

12. Page d'accueil

La homepage devra fonctionner comme une vitrine digitale.

Hero principal

Grande image lifestyle.

Exemple :

Votre style. Votre intérieur. Votre univers.

Sous-titre :

Découvrez nos nouvelles collections mode & maison.

Boutons :

Découvrir la collection

Voir les nouveautés

13. Section catégories

Une section visuelle permettra d'accéder rapidement aux univers.

Exemple :

FEMME

HOMME

CHAUSSURES

ACCESSOIRES

MAISON

Chaque catégorie devra être représentée par une grande image.

14. Section Nouveautés

Titre :

Nouveautés

Sous-titre :

Découvrez les dernières pièces ajoutées à notre collection.

Afficher les produits les plus récents.

15. Section Femme

Grande section éditoriale :

Collection Femme

avec :

- image principale ;
 - produits sélectionnés ;
 - bouton "Découvrir Femme".
-

16. Section Homme

Même principe :

Collection Homme

avec une présentation visuelle masculine.

17. Section Chaussures

Présentation des dernières chaussures :

- sneakers ;
- sandales ;
- mocassins ;
- escarpins.

18. Section Maison & Décoration

Cette section devra avoir une identité visuelle légèrement différente.

Objectif :

Créer une ambiance **maison / lifestyle / élégance**.

Exemple :

Transformez votre intérieur.

Présenter :

- vases ;
- miroirs ;
- bougies ;
- coussins ;
- luminaires ;
- objets décoratifs.

19. Promotions

Une section spécifique permettra de mettre en avant les offres.

Exemple :

SALE

Jusqu'à -50%

Chaque produit pourra afficher :

Ancien prix : 35 000 FCFA
Nouveau prix : 25 000 FCFA
-29%

20. Catalogue

La page catalogue devra permettre une navigation très fluide.

Affichage

- grille 2 colonnes mobile ;
 - grille 3 colonnes tablette ;
 - grille 4 colonnes desktop.
-

21. Filtres

Les filtres devront dépendre de la catégorie.

Filtres généraux

- prix ;
- disponibilité ;
- nouveautés ;
- promotions.

Femme / Homme

- taille ;
- couleur ;
- matière ;
- coupe.

Chaussures

- pointure ;
- couleur ;
- matière.

Maison

- couleur ;

- matière ;
 - dimensions ;
 - pièce.
-

22. Tri

Options :

- Pertinence ;
 - Nouveautés ;
 - Prix croissant ;
 - Prix décroissant ;
 - Meilleures ventes.
-

23. Carte produit

Chaque produit devra présenter :

- photo ;
- badge ;
- nom ;
- prix ;
- ancien prix ;
- réduction ;
- couleur disponible ;
- bouton favori.

Exemple :



Robe longue satinée
35 000 FCFA

24. Effet au survol

Desktop :

Lorsque l'utilisateur passe la souris sur un produit :

- deuxième image ;
 - apparition du bouton "Ajouter au panier" ;
 - animation légère.
-

25. Page produit

La page produit devra être très visuelle.

Galerie

- image principale ;
 - miniatures ;
 - zoom ;
 - plusieurs photos ;
 - images lifestyle.
-

26. Informations produit

Afficher :

- nom ;
 - référence ;
 - prix ;
 - ancien prix ;
 - réduction ;
 - description ;
 - disponibilité ;
 - matière ;
 - entretien ;
 - livraison.
-

27. Variantes

Les produits vestimentaires devront pouvoir avoir plusieurs variantes.

Exemple :

Couleur

- Noir
- Blanc
- Beige
- Rouge

Taille

- XS
- S
- M
- L
- XL
- XXL

Pour les chaussures :

- 36
- 37
- 38
- 39
- 40
- 41
- 42
- 43
- 44
- 45

28. Gestion intelligente des variantes

Lorsqu'une taille ou une couleur est indisponible :

Elle devra être automatiquement désactivée.

Exemple :

S M L XL
• • × •

Le client comprendra immédiatement que la taille L n'est plus disponible.

29. Guide des tailles

Pour les vêtements, une fonctionnalité :

Guide des tailles

permettra d'afficher un tableau.

Exemple :

Taille	Poitrine	Taille	Hanches
S	88-92	68-72	92-96
M	92-96	72-76	96-100
L	96-100	76-80	100-104
XL	100-106	80-86	104-110

Les valeurs seront configurables depuis l'administration.

30. Ajouter au panier

Le bouton principal devra être très visible :

AJOUTER AU PANIER

Après ajout :

- animation ;
- notification ;
- mise à jour du compteur panier.

31. Achat rapide

Un bouton pourra permettre :

Acheter maintenant

et envoyer directement vers le checkout.

32. Favoris

Le client pourra sauvegarder ses produits favoris.

Le système devra permettre :

- ajouter ;
- supprimer ;
- consulter ;
- ajouter au panier.

Pour les utilisateurs non connectés, les favoris pourront être conservés temporairement côté navigateur.

33. Panier

Le panier devra être accessible depuis le header.

Il pourra s'ouvrir sous forme de drawer.

Afficher :

- produit ;
- image ;
- variante ;
- taille ;
- couleur ;
- quantité ;
- prix ;
- suppression.

Résumé :

Sous-total
Livraison
Réduction
TOTAL

34. Checkout

Le checkout devra être très simple.

Étape 1

Informations personnelles :

- nom ;
- prénom ;
- téléphone ;
- email.

Étape 2

Adresse :

- région ;
- ville ;
- quartier ;
- adresse ;
- instructions.

Étape 3

Livraison.

Étape 4

Paiement.

Étape 5

Confirmation.

35. Livraison

Le système devra gérer la livraison partout au Sénégal.

Possibilité de définir :

- zone ;
- délai ;
- coût.

Exemple :

Dakar

Livraison : 2 000 FCFA

Thiès

Livraison : 3 000 FCFA

Autres régions

Tarif configurable

Les tarifs seront administrables.

36. Paiement

Le système devra être conçu pour intégrer les moyens de paiement adaptés au marché sénégalais.

Exemples possibles :

- Wave ;
- Orange Money ;
- paiement par carte ;
- paiement à la livraison si disponible ;
- autres solutions de paiement.

Les intégrations exactes seront définies avant le développement.

37. Compte client

Le client pourra :

- créer un compte ;
- se connecter ;

- modifier son profil ;
 - modifier son mot de passe ;
 - gérer ses adresses ;
 - consulter ses commandes ;
 - consulter ses favoris ;
 - suivre ses commandes.
-

38. Suivi de commande

Timeline :

Commande reçue



Confirmée



En préparation



Expédiée



En livraison



Livrée

39. Avis clients

Une évolution importante pourra permettre aux clients de noter les produits.

Système :

★★★★★

et commentaire.

L'administrateur pourra modérer les avis.

40. Back-office

L'administration Laravel devra être conçue avec la même exigence UX que le frontend.

Dashboard :

- chiffre d'affaires ;
 - commandes ;
 - clients ;
 - produits ;
 - stock ;
 - produits populaires ;
 - ventes récentes.
-

41. Gestion des produits

L'administrateur pourra :

- ajouter ;
 - modifier ;
 - supprimer ;
 - désactiver ;
 - mettre en promotion ;
 - mettre en avant ;
 - gérer le stock ;
 - gérer les variantes ;
 - gérer les images.
-

42. Création d'un produit

Formulaire :

Nom

Catégorie

Sous-catégorie

Description

Prix

Ancien prix

SKU

Stock

Images

Couleurs

Tailles

Matière

Dimensions
Poids
Nouveauté
Promotion
Produit vedette

Les champs devront être dynamiques selon la catégorie.

Par exemple :

Pour une robe :

Taille
Couleur
Matière
Coupe

Pour un vase :

Dimensions
Matière
Couleur

43. Gestion des catégories

L'administrateur pourra créer :

Femme

- └─ Robes
- └─ Tops
- └─ Pantalons
- └─ Ensembles

Homme

- └─ T-shirts
- └─ Chemises
- └─ Jeans
- └─ Pantalons

Chaussures

- └─ Sneakers
- └─ Sandales
- └─ Mocassins

Accessoires

- └ Sacs
- └ Bijoux
- └ Lunettes

Maison

- └ Décoration
- └ Salon
- └ Chambre
- └ Luminaires

44. Gestion des commandes

L'administrateur pourra :

- consulter les commandes ;
- rechercher une commande ;
- filtrer par statut ;
- voir les détails ;
- modifier le statut ;
- confirmer ;
- annuler ;
- marquer comme expédiée ;
- marquer comme livrée.

45. Gestion du stock

Le stock devra être géré au niveau des variantes.

Exemple :

Robe noire

S → 5

M → 10

L → 0

XL → 3

Le site affichera :

L : Rupture de stock

46. Gestion des promotions

L'administration permettra de créer :

- réduction en pourcentage ;
 - réduction fixe ;
 - promotion par catégorie ;
 - promotion sur produit ;
 - code promo ;
 - date de début ;
 - date de fin.
-

47. Codes promotionnels

Exemple :

Code : KHALIL20

Réduction : 20%

Minimum : 50 000 FCFA

Expiration : 31/12/2026

48. Gestion de la homepage

L'administrateur devra pouvoir modifier :

- hero ;
- bannières ;
- produits vedettes ;
- nouveautés ;
- promotions ;
- collections ;
- sections maison.

L'objectif est de permettre à KhalilShop de modifier son contenu commercial **sans intervention du développeur**.

49. Gestion des images

Les images seront stockées sur Cloudinary.

Fonctionnalités :

- upload ;
- optimisation ;
- compression ;
- transformation ;
- recadrage ;
- plusieurs tailles ;
- suppression.

Le système devra conserver :

- URL ;
 - public ID ;
 - ordre ;
 - texte alternatif.
-

50. Base de données Laravel

Tables principales :

```
users
categories
subcategories
products
product_variants
product_images
colors
sizes
orders
order_items
addresses
favorites
reviews
coupons
coupon_usages
promotions
banners
payments
deliveries
```

51. Relations Eloquent

Exemple :

Category

- |
- └─ hasMany Products

Product

- └─ belongsTo Category
- └─ hasMany Variants
- └─ hasMany Images
- └─ hasMany Reviews
- └─ belongsToMany Favorites

ProductVariant

- └─ belongsTo Product
- └─ belongsTo Size
- └─ belongsTo Color

User

- └─ hasMany Orders
- └─ hasMany Addresses
- └─ hasMany Favorites
- └─ hasMany Reviews

Order

- └─ belongsTo User
- └─ hasMany OrderItems

52. Architecture Laravel

Le projet devra respecter une architecture MVC propre.

app/

- └─ Models
- └─ Http
 - | └─ Controllers
 - | └─ Requests
 - | └─ Middleware
- └─ Services

Les traitements complexes devront être placés dans des **Services** plutôt que directement dans les contrôleurs.

53. Frontend Tailwind CSS

Tailwind CSS sera utilisé pour construire l'ensemble de l'interface.

Composants réutilisables :

Navbar
MegaMenu
Hero
ProductCard
ProductGrid
ProductGallery
FilterSidebar
SearchBar
CartDrawer
Modal
Toast
Badge
Button
Input
Select
Pagination
Breadcrumb
Footer

54. Alpine.js

Alpine.js pourra gérer les interactions frontend légères :

- menu mobile ;
- mega menu ;
- filtres ;
- modal ;
- panier drawer ;
- galerie ;

- sélection variante ;
- notifications ;
- accordéons ;
- recherche instantanée.

Cela permettra d'éviter d'ajouter inutilement un framework frontend lourd.

55. Responsive Design

Le site devra être parfaitement responsive.

Mobile

2 produits / ligne

Tablette

3 produits / ligne

Desktop

4 produits / ligne

Grand écran

Possibilité :

5 produits / ligne

56. Performance

Objectif : expérience extrêmement rapide.

Techniques :

- lazy loading ;
- Cloudinary ;
- WebP/AVIF ;
- responsive images ;
- pagination ;
- cache Laravel ;

- eager loading Eloquent ;
 - optimisation des requêtes ;
 - Vite ;
 - minification ;
 - CDN.
-

57. SEO

Chaque produit devra disposer de :

- slug ;
- title ;
- meta description ;
- image SEO ;
- URL canonique ;
- Open Graph.

Exemple :

`/khalilshop/femme/robes/robe-satin-beige`

58. Sécurité

Laravel devra assurer :

- CSRF ;
 - validation serveur ;
 - hashage des mots de passe ;
 - protection XSS ;
 - protection SQL Injection ;
 - autorisation par Policies ;
 - validation des uploads ;
 - protection des routes admin ;
 - rate limiting ;
 - HTTPS.
-

59. Gestion des rôles

Client

Accès :

- catalogue ;
- panier ;
- compte ;
- commandes ;
- favoris.

Administrateur

Accès complet.

Gestionnaire

Accès possible à :

- produits ;
 - stocks ;
 - commandes.
-

60. Notifications

Notifications pour :

Client

- commande reçue ;
- commande confirmée ;
- commande expédiée ;
- commande livrée ;
- paiement confirmé.

Administrateur

- nouvelle commande ;
 - paiement ;
 - stock faible.
-

61. Emails

Emails automatiques :

Bienvenue chez KhalilShop
Confirmation de commande
Paielement confirmé
Commande expédiée
Commande livrée
Réinitialisation mot de passe

62. Analytics

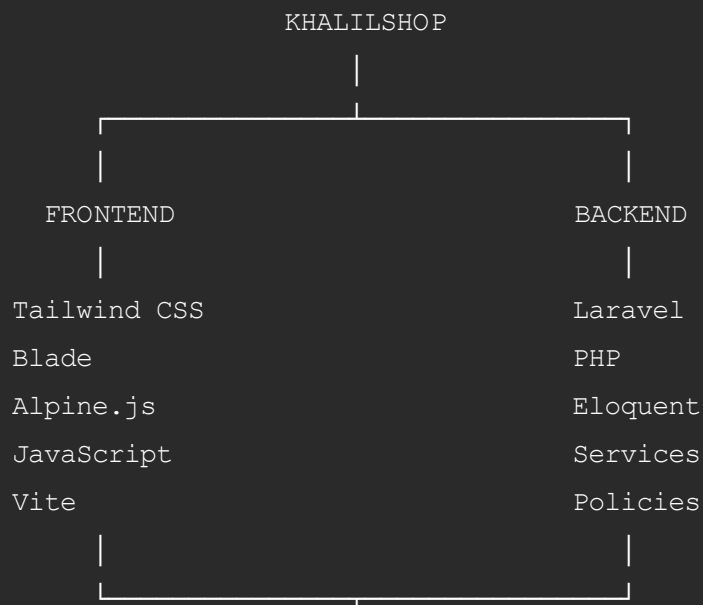
Le site pourra intégrer :

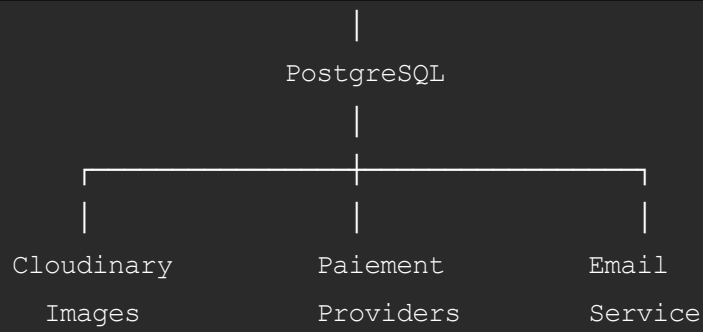
- Google Analytics ;
- Meta Pixel ;
- Google Search Console.

Événements :

view_item
search
add_to_cart
begin_checkout
purchase
sign_up

63. Architecture technique finale





64. Phases de développement

Phase 1 — Analyse

- validation du catalogue ;
- catégories ;
- variantes ;
- règles commerciales ;
- livraison ;
- paiement.

Phase 2 — UX/UI

- wireframes ;
- architecture ;
- design system ;
- maquettes desktop ;
- maquettes mobile.

Phase 3 — Laravel

- installation ;
- migrations ;
- modèles ;
- relations Eloquent ;
- authentification ;
- rôles ;
- sécurité.

Phase 4 — Frontend

- header ;
- mega menu ;
- homepage ;

- catalogue ;
- filtres ;
- produit ;
- panier ;
- checkout.

Phase 5 — Administration

- dashboard ;
- produits ;
- catégories ;
- stock ;
- commandes ;
- clients ;
- promotions ;
- homepage.

Phase 6 — Intégrations

- Cloudinary ;
- paiement ;
- emails ;
- analytics.

Phase 7 — Tests

- fonctionnels ;
- responsive ;
- sécurité ;
- performance ;
- compatibilité navigateurs.

Phase 8 — Déploiement

- serveur ;
 - domaine ;
 - SSL ;
 - base de données ;
 - Cloudinary ;
 - variables d'environnement ;
 - sauvegardes.
-

65. Critères de réussite

KhalilShop devra être considéré comme réussi lorsque :

- le design est moderne et premium ;
 - la navigation est intuitive ;
 - le catalogue est facilement exploitable ;
 - les variantes sont correctement gérées ;
 - le panier fonctionne parfaitement ;
 - le checkout est simple ;
 - la gestion des stocks est fiable ;
 - les commandes sont correctement suivies ;
 - l'administration est facile à utiliser ;
 - le site est parfaitement responsive ;
 - les pages sont rapides ;
 - le référencement de base est correctement configuré ;
 - le site est sécurisé.
-

66. Vision finale

KhalilShop ne devra pas ressembler à une simple boutique en ligne générique.

L'objectif est de créer une véritable **marque digitale Mode & Lifestyle**.

L'expérience devra donner l'impression d'entrer dans une boutique moderne où les univers sont clairement séparés :

FEMME

HOMME

CHAUSSURES

ACCESSOIRES

MAISON & DÉCORATION

avec une identité visuelle cohérente entre tous les univers.

Le site devra combiner :

tout en restant rapide, simple et adapté aux habitudes d'achat des clients au Sénégal.

67. Stack technologique définitive

Frontend

- └─ Laravel Blade
- └─ Tailwind CSS
- └─ Alpine.js
- └─ JavaScript
- └─ Vite

Backend

- └─ Laravel
- └─ PHP
- └─ Eloquent ORM
- └─ Laravel Authentication
- └─ Policies / Gates
- └─ Form Requests
- └─ Services

Database

- └─ PostgreSQL

Images

- └─ Cloudinary

Versioning

- └─ Git / GitHub

Infrastructure

- └─ Serveur Linux + HTTPS

KhalilShop devra donc être développé comme une plateforme e-commerce moderne, évolutive et réellement orientée conversion, avec une architecture Laravel propre et une interface Tailwind CSS premium.

Oui. KhalilShop sera un site e-commerce B2C (Business to Consumer).

Cela signifie que **KhalilShop vend directement ses produits aux consommateurs particuliers**, et non principalement à des entreprises ou revendeurs.

Positionnement B2C de KhalilShop

KhalilShop → Client particulier

Le parcours principal sera donc :

Découverte → Recherche → Produit → Panier → Commande → Paiement → Livraison

Les fonctionnalités prioritaires seront :

-  **Compte client**
-  **Catalogue produits**
-  **Recherche**
-  **Filtres et catégories**
-  **Favoris**
-  **Panier**
-  **Commande**
-  **Paiement**
-  **Livraison**
-  **Notifications**
-  **Avis clients**
-  **Codes promo**
-  **Expérience mobile-first**

Et les catégories principales restent :

Femme | Homme | Chaussures | Accessoires | Maison & Décoration

Dans le cahier des charges, je recommande donc de remplacer « **Type : Site e-commerce B2C** » par « **Modèle commercial : B2C — Business to Consumer** » et de

supprimer toute fonctionnalité orientée B2B, comme les comptes entreprises, tarifs grossistes ou commandes professionnelles.

1. Réaliser un annuaire de contacts permettant de gérer : prénom, nom, adresse e-mail, numéro de téléphone.
2. Implémenter les fonctionnalités suivantes : ajouter, afficher la liste, consulter, modifier et supprimer une personne. tu es un expert en developement laravel et tailwind css avec plus de 15 ans d experiences

Oui. Pour cet exercice, je te conseille de réaliser un **CRUD Laravel complet** avec **Tailwind CSS**, en respectant une architecture Laravel propre.

L'application sera un **annuaire de contacts** permettant de gérer :

- Prénom
- Nom
- Adresse e-mail
- Numéro de téléphone

Et les 5 opérations :

1. **Ajouter** une personne
2. **Afficher la liste** des personnes
3. **Consulter** une personne
4. **Modifier** une personne
5. **Supprimer** une personne

1. Architecture du projet

On va utiliser :

```
Laravel
├─ Model      → Contact
├─ Migration  → contacts
├─ Controller → ContactController
├─ Form Request → validation
├─ Routes     → web.php
└─ Views      → Blade + Tailwind CSS
```

Les routes seront :

```
GET    /contacts      → liste
GET    /contacts/create → formulaire d'ajout
```

POST	/contacts	→ enregistrer
GET	/contacts/{contact}	→ consulter
GET	/contacts/{contact}/edit	→ formulaire de modification
PUT	/contacts/{contact}	→ modifier
DELETE	/contacts/{contact}	→ supprimer

2. Créer le projet Laravel

Si le projet n'existe pas encore :

Bash

```
laravel new annuaire
```

Puis :

Bash

```
cd annuaire
```

Installer les dépendances frontend :

Bash

```
npm install  
npm run build
```

Pour le développement :

Bash

```
npm run dev
```

Dans un deuxième terminal :

Bash

```
php artisan serve
```

3. Configurer la base de données

Dans `.env`, configure ta base PostgreSQL ou MySQL.

Exemple PostgreSQL :

env

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=annuaire
DB_USERNAME=postgres
DB_PASSWORD=ton_mot_de_passe
```

Puis :

Bash

```
php artisan migrate
```

4. Créer le modèle et la migration

Commande :

Bash

```
php artisan make:model Contact -m
```

Laravel va créer :

```
app/Models/Contact.php
database/migrations/xxxx_xx_xx_XXXXXX_create_contacts_table.php
```

5. Migration contacts

Dans :

```
database/migrations/xxxx_xx_xx_XXXXXX_create_contacts_table.php
```

mets :

PHP

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('contacts', function (Blueprint $table) {
            $table->id();
            $table->string('prenom');
            $table->string('nom');
            $table->string('email')->unique();
            $table->string('telephone');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('contacts');
    }
};

```

Puis :

Bash

```
php artisan migrate
```

6. Configurer le modèle Contact

Dans :

```
app/Models/Contact.php
```

Mets :

PHP

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Contact extends Model
{
    protected $fillable = [
        'prenom',
        'nom',

```

```
'email',  
'telephone',  
];  
}
```

`$fillable` permet à Laravel d'autoriser l'affectation en masse de ces champs.

7. Créer le Controller

Commande :

Bash

```
php artisan make:controller ContactController --resource
```

Laravel crée :

```
app/Http/Controllers/ContactController.php
```

Le controller devra gérer les 7 méthodes CRUD de Laravel :

```
index()  
create()  
store()  
show()  
edit()  
update()  
destroy()
```

8. ContactController

Voici une version propre :

PHP

```
<?php  
  
namespace App\Http\Controllers;  
  
use App\Models>Contact;  
use Illuminate\Http\Request;  
  
class ContactController extends Controller  
{  
    /**
```



```

/**
 * Modifier un contact.
 */
public function update(Request $request, Contact $contact)
{
    $validated = $request->validate([
        'prenom' => ['required', 'string', 'max:100'],
        'nom' => ['required', 'string', 'max:100'],
        'email' => [
            'required',
            'email',
            'max:255',
            'unique:contacts,email,' . $contact->id,
        ],
        'telephone' => ['required', 'string', 'max:30'],
    ]);

    $contact->update($validated);

    return redirect()
        ->route('contacts.index')
        ->with('success', 'Contact modifié avec succès.');
```

```

/**
 * Supprimer un contact.
 */
public function destroy(Contact $contact)
{
    $contact->delete();

    return redirect()
        ->route('contacts.index')
        ->with('success', 'Contact supprimé avec succès.');
```

9. Comprendre le Route Model Binding

Cette partie est importante en Laravel.

Dans :

PHP

```
public function show(Contact $contact)
```

Laravel comprend automatiquement que :


```
/contacts/5
```

doit rechercher :

PHP

```
Contact::findOrFail(5);
```

Tu n'as donc pas besoin de faire manuellement :

PHP

```
$contact = Contact::findOrFail($id);
```

C'est ce qu'on appelle le **Route Model Binding**.

10. Définir les routes

Dans :

```
routes/web.php
```

mets :

PHP

```
<?php

use App\Http\Controllers\ContactController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return redirect()->route('contacts.index');
});

Route::resource('contacts', ContactController::class);
```

Une seule ligne :

PHP

```
Route::resource('contacts', ContactController::class);
```

génère automatiquement les 7 routes du CRUD.

Pour vérifier :

Bash

```
php artisan route:list
```

Tu devrais obtenir notamment :

```
GET      contacts
GET      contacts/create
POST     contacts
GET      contacts/{contact}
GET      contacts/{contact}/edit
PUT/PATCH contacts/{contact}
DELETE   contacts/{contact}
```

11. Structure des vues

Crée :

```
resources/views/
├── contacts/
│   ├── index.blade.php
│   ├── create.blade.php
│   ├── show.blade.php
│   └── edit.blade.php
```

Et un layout :

```
resources/views/
├── layouts/
│   └── app.blade.php
```

12. Layout principal

```
resources/views/layouts/app.blade.php
```

blade

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
```

```

<meta name="viewport"
      content="width=device-width, initial-scale=1.0">

<title>{{ $title ?? 'Annuaire de contacts' }}</title>

@vite(['resources/css/app.css', 'resources/js/app.js'])
</head>

<body class="min-h-screen bg-gray-50 text-gray-900">

  <nav class="border-b bg-white">
    <div class="mx-auto flex max-w-7xl items-center justify-between px-6 py-4">

      <a href="{{ route('contacts.index') }}"
        class="text-xl font-bold">
        Annuaire
      </a>

      <a href="{{ route('contacts.create') }}"
        class="rounded-lg bg-blue-600 px-4 py-2 text-sm font-medium text-white tra
        + Ajouter
      </a>

    </div>
  </nav>

  <main class="mx-auto max-w-7xl px-6 py-8">

    @if(session('success'))
      <div class="mb-6 rounded-lg border border-green-200 bg-green-50 px-4 py-3 tex
        {{ session('success') }}
      </div>
    @endif

    @yield('content')

  </main>

</body>
</html>

```

13. Page liste

```
resources/views/contacts/index.blade.php
```

blade

```
@extends('layouts.app')
```

```

@section('content')

<div class="mb-8 flex items-center justify-between">

    <div>
        <h1 class="text-3xl font-bold">
            Annuaire de contacts
        </h1>

        <p class="mt-2 text-gray-500">
            Gérez facilement vos contacts.
        </p>
    </div>

    <a href="{{ route('contacts.create') }}"
        class="rounded-lg bg-blue-600 px-5 py-3 font-medium text-white hover:bg-blue-700">
        Ajouter un contact
    </a>

</div>

<div class="overflow-hidden rounded-xl border border-gray-200 bg-white shadow-sm">

    <table class="w-full">

        <thead class="bg-gray-50">
            <tr>
                <th class="px-6 py-4 text-left text-sm font-semibold">
                    Prénom
                </th>

                <th class="px-6 py-4 text-left text-sm font-semibold">
                    Nom
                </th>

                <th class="px-6 py-4 text-left text-sm font-semibold">
                    Email
                </th>

                <th class="px-6 py-4 text-left text-sm font-semibold">
                    Téléphone
                </th>

                <th class="px-6 py-4 text-right text-sm font-semibold">
                    Actions
                </th>
            </tr>
        </thead>

        <tbody class="divide-y divide-gray-100">

            @forelse($contacts as $contact)

```

```

<tr class="hover:bg-gray-50">

  <td class="px-6 py-4">
    {{ $contact->prenom }}
  </td>

  <td class="px-6 py-4 font-medium">
    {{ $contact->nom }}
  </td>

  <td class="px-6 py-4 text-gray-600">
    {{ $contact->email }}
  </td>

  <td class="px-6 py-4 text-gray-600">
    {{ $contact->telephone }}
  </td>

  <td class="px-6 py-4">

    <div class="flex justify-end gap-2">

      <a href="{{ route('contacts.show', $contact) }}"
        class="rounded-lg bg-gray-100 px-3 py-2 text-sm hover:bg-gray-100"
        >Voir
      </a>

      <a href="{{ route('contacts.edit', $contact) }}"
        class="rounded-lg bg-blue-50 px-3 py-2 text-sm text-blue-600"
        >Modifier
      </a>

      <form action="{{ route('contacts.destroy', $contact) }}"
        method="POST"
        onsubmit="return confirm('Voulez-vous vraiment supprimer ce contact?');"
        @csrf
        @method('DELETE')
      >
        <button type="submit"
          class="rounded-lg bg-red-50 px-3 py-2 text-sm text-red-600"
          >Supprimer
        </button>
      </form>
    </div>
  </td>
</tr>

```

```

        @empty

            <tr>
                <td colspan="5"
                    class="px-6 py-12 text-center text-gray-500">

                    Aucun contact enregistré.

                </td>
            </tr>

        @endforelse

    </tbody>

</table>

</div>

<div class="mt-6">
    {{ $contacts->links() }}
</div>

```

14. Formulaire d'ajout

resources/views/contacts/create.blade.php

blade

```

@extends('layouts.app')

@section('content')

<div class="mx-auto max-w-2xl">

    <div class="mb-8">
        <h1 class="text-3xl font-bold">
            Ajouter un contact
        </h1>

        <p class="mt-2 text-gray-500">
            Ajoutez une nouvelle personne à votre annuaire.
        </p>
    </div>

    <form action="{{ route('contacts.store') }}"
        method="POST"

```



```

<label for="email"
      class="mb-2 block text-sm font-medium">
  Adresse e-mail
</label>

<input
  type="email"
  id="email"
  name="email"
  value="{{ old('email') }}"
  class="w-full rounded-lg border-gray-300 px-4 py-3 focus:border-blue-500
>

@error('email')
  <p class="mt-1 text-sm text-red-600">
    {{ $message }}
  </p>
@enderror

</div>

<div class="mt-6">

  <label for="telephone"
        class="mb-2 block text-sm font-medium">
    Numéro de téléphone
  </label>

  <input
    type="text"
    id="telephone"
    name="telephone"
    value="{{ old('telephone') }}"
    class="w-full rounded-lg border-gray-300 px-4 py-3 focus:border-blue-500
  >

  @error('telephone')
    <p class="mt-1 text-sm text-red-600">
      {{ $message }}
    </p>
  @enderror

</div>

<div class="mt-8 flex justify-end gap-3">

  <a href="{{ route('contacts.index') }}"
    class="rounded-lg border border-gray-300 px-5 py-3 font-medium hover:bg-gr
    Annuler
  </a>

  <button

```



```

        type="submit"
        class="rounded-lg bg-blue-600 px-5 py-3 font-medium text-white hover:bg-b
        Enregistrer
    </button>

</div>

</form>

</div>

```

15. Page consultation

resources/views/contacts/show.blade.php

blade

```

@extends('layouts.app')

@section('content')

<div class="mx-auto max-w-2xl">

    <div class="mb-8 flex items-center justify-between">

        <div>
            <h1 class="text-3xl font-bold">
                {{ $contact->prenom }} {{ $contact->nom }}
            </h1>

            <p class="mt-2 text-gray-500">
                Informations du contact
            </p>
        </div>

        <a href="{{ route('contacts.edit', $contact) }}"
            class="rounded-lg bg-blue-600 px-5 py-3 font-medium text-white hover:bg-blue-7
            Modifier
        </a>

    </div>

    <div class="overflow-hidden rounded-xl border border-gray-200 bg-white shadow-sm">

        <div class="border-b px-6 py-5">
            <p class="text-sm text-gray-500">
                Prénom
            </p>

```

```

        <p class="mt-1 text-lg font-medium">
            {{ $contact->prenom }}
        </p>
    </div>

    <div class="border-b px-6 py-5">
        <p class="text-sm text-gray-500">
            Nom
        </p>

        <p class="mt-1 text-lg font-medium">
            {{ $contact->nom }}
        </p>
    </div>

    <div class="border-b px-6 py-5">
        <p class="text-sm text-gray-500">
            Adresse e-mail
        </p>

        <p class="mt-1 text-lg font-medium">
            {{ $contact->email }}
        </p>
    </div>

    <div class="px-6 py-5">
        <p class="text-sm text-gray-500">
            Téléphone
        </p>

        <p class="mt-1 text-lg font-medium">
            {{ $contact->telephone }}
        </p>
    </div>

</div>

<div class="mt-6">
    <a href="{{ route('contacts.index') }}"
        class="text-sm font-medium text-gray-600 hover:text-gray-900">
        ← Retour à la liste
    </a>
</div>

</div>

@endsection

```

16. Page modification

Le plus simple est de réutiliser le même formulaire.

resources/views/contacts/edit.blade.php

blade

```
@extends('layouts.app')

@section('content')

<div class="mx-auto max-w-2xl">

    <div class="mb-8">

        <h1 class="text-3xl font-bold">
            Modifier le contact
        </h1>

        <p class="mt-2 text-gray-500">
            Modifiez les informations de {{ $contact->prenom }} {{ $contact->nom }}.
        </p>

    </div>

    <form action="{{ route('contacts.update', $contact) }}"
        method="POST"
        class="rounded-xl border border-gray-200 bg-white p-6 shadow-sm">

        @csrf
        @method('PUT')

        <div class="grid gap-6 md:grid-cols-2">

            <div>

                <label class="mb-2 block text-sm font-medium">
                    Prénom
                </label>

                <input
                    type="text"
                    name="prenom"
                    value="{{ old('prenom', $contact->prenom) }}"
                    class="w-full rounded-lg border-gray-300 px-4 py-3"
                >

                @error('prenom')
                    <p class="mt-1 text-sm text-red-600">
                        {{ $message }}
                    </p>
                @enderror

            </div>

        </div>

    </form>

</div>
```

```

</div>

<div>

    <label class="mb-2 block text-sm font-medium">
        Nom
    </label>

    <input
        type="text"
        name="nom"
        value="{{ old('nom', $contact->nom) }}"
        class="w-full rounded-lg border-gray-300 px-4 py-3"
    >

    @error('nom')
        <p class="mt-1 text-sm text-red-600">
            {{ $message }}
        </p>
    @enderror

</div>

</div>

<div class="mt-6">

    <label class="mb-2 block text-sm font-medium">
        Adresse e-mail
    </label>

    <input
        type="email"
        name="email"
        value="{{ old('email', $contact->email) }}"
        class="w-full rounded-lg border-gray-300 px-4 py-3"
    >

    @error('email')
        <p class="mt-1 text-sm text-red-600">
            {{ $message }}
        </p>
    @enderror

</div>

<div class="mt-6">

    <label class="mb-2 block text-sm font-medium">
        Numéro de téléphone
    </label>

```

```

        <input
            type="text"
            name="telephone"
            value="{{ old('telephone', $contact->telephone) }}"
            class="w-full rounded-lg border-gray-300 px-4 py-3"
        >

        @error('telephone')
            <p class="mt-1 text-sm text-red-600">
                {{ $message }}
            </p>
        @enderror

    </div>

    <div class="mt-8 flex justify-end gap-3">

        <a href="{{ route('contacts.index') }}"
            class="rounded-lg border px-5 py-3">
            Annuler
        </a>

        <button
            type="submit"
            class="rounded-lg bg-blue-600 px-5 py-3 font-medium text-white hover:bg-b
            Enregistrer les modifications
        </button>

    </div>

</form>

</div>

```

17. Le CRUD est maintenant complet

Tu as :

Fonctionnalité

Laravel

Ajouter

`store()`

Afficher la liste

`index()`

Consulter

`show()`

Modifier

`update()`

Fonctionnalité

Laravel

Supprimer

`destroy()`

Et les pages :

```
/contacts  
/contacts/create  
/contacts/{contact}  
/contacts/{contact}/edit
```

18. Ce que tu dois comprendre dans cet exercice

Cet exercice est très important parce qu'il te fait pratiquer les fondamentaux de Laravel :

Migration

→ création de la structure de la base de données

Model

→ représentation de la table `contacts`

Eloquent

→ communication avec la base de données

Controller

→ logique de l'application

Routes

→ association URL → Controller

Blade

→ affichage HTML

Tailwind CSS

→ interface utilisateur

Validation

→ contrôle des données reçues

Route Model Binding

→ récupération automatique du contact

Resource Controller

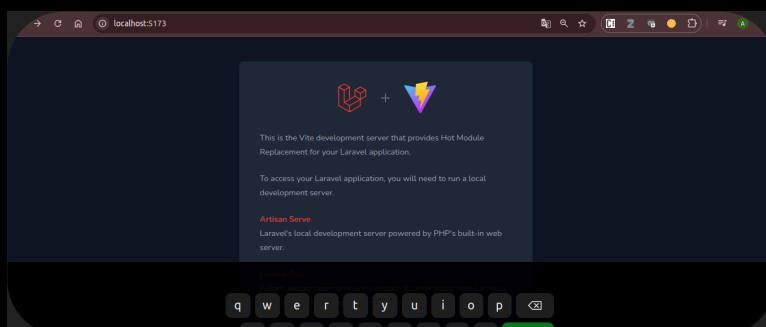
→ architecture CRUD standard de Laravel

Le schéma mental à retenir est :

```
Utilisateur
  ↓
Route
  ↓
Controller
  ↓
Validation
  ↓
Model
  ↓
Eloquent
  ↓
Database
  ↓
Controller
  ↓
Blade
  ↓
Tailwind CSS
  ↓
Utilisateur
```

C'est exactement le type d'architecture que tu dois maîtriser avant de passer à des applications Laravel plus complexes comme **KhalilShop**.

hier 20:40



j ai toujours cette page

Oui, je vois exactement le problème sur ta capture. 👍

Ton Laravel fonctionne probablement, mais tu es actuellement sur le serveur Vite :

```
localhost:5173.
```

La page que tu vois :

```
"This is the Vite development server that provides Hot Module Replacement..."
```

n'est pas la page de ton application Laravel.

1. Tu dois avoir 2 serveurs

Terminal 1 — Laravel

Va dans le dossier de ton projet :

Bash

```
cd /var/www/html/annuaire
```

Puis lance :

Bash

```
php artisan serve
```

Tu devrais obtenir quelque chose comme :

```
INFO  Server running on [http://127.0.0.1:8000].
```

2. Terminal 2 — Vite

Dans un autre terminal :

Bash

```
cd /var/www/html/annuaire
npm run dev
```

Tu auras :

```
VITE v...
→ Local: http://localhost:5173/
```

Ne mets pas `localhost:5173` dans ton navigateur pour afficher Laravel.

3. Ouvre Laravel dans le navigateur

Va sur :

```
http://127.0.0.1:8000
```

ou :

```
http://localhost:8000
```

Tu dois alors arriver sur ton application Laravel.

Comme nous avons défini cette route :

PHP

```
Route::get('/', function () {  
    return redirect()->route('contacts.index');  
});
```

tu devrais être automatiquement redirigée vers :

```
http://localhost:8000/contacts
```

et voir ton **annuaire de contacts**.

4. Vérifie également tes routes

Dans le terminal :

Bash

```
php artisan route:list
```

Tu dois notamment avoir :

```
GET|HEAD  /  
GET|HEAD  contacts  
GET|HEAD  contacts/create  
POST      contacts  
GET|HEAD  contacts/{contact}  
GET|HEAD  contacts/{contact}/edit  
PUT|PATCH contacts/{contact}  
DELETE    contacts/{contact}
```

5. Si `php artisan serve` affiche une erreur

Fais :

Bash

```
php artisan serve
```

et copie-moi exactement tout ce qui apparaît dans le terminal.

Mais d'après ta capture, le problème principal est simplement celui-ci :

✗ localhost:5173

↓

Vite

✓ localhost:8000

↓

Laravel

↓

Annuaire

Important

Tu **dois laisser** `npm run dev` **tourner** si ton application utilise Tailwind/Vite.

Donc au final :

Terminal 1

Bash

```
php artisan serve
```

Terminal 2

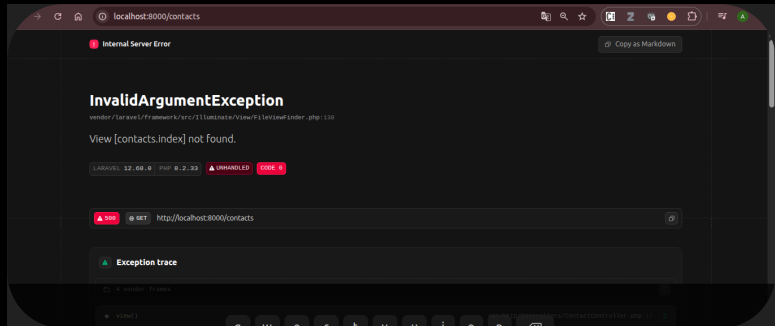
Bash

```
npm run dev
```

Navigateur :

```
http://localhost:8000
```

Si après ça tu vois encore la page Laravel/Vite, envoie-moi **la sortie de** `php artisan serve` + `php artisan route:list`, et je te guiderai étape par étape.



Parfait, maintenant **Laravel fonctionne correctement.** ✅

L'erreur affichée est très précise :

View [contacts.index] not found.

Cela signifie que Laravel arrive bien à exécuter `ContactController@index`, mais qu'il cherche cette vue :

```
resources/views/contacts/index.blade.php
```

et qu'elle **n'existe pas encore**, ou qu'elle n'est pas au bon endroit.

1. Vérifie la structure de ton projet

Dans ton terminal, place-toi dans ton projet :

Bash

```
cd /var/www/html/annuaire
```

Puis :

Bash

```
ls resources/views
```

Ensuite :

Bash

```
ls resources/views/contacts
```

Si tu obtiens :

```
ls: cannot access 'resources/views/contacts': No such file or directory
```

c'est normal : il faut créer les dossiers.

2. Crée le dossier `contacts`

Exécute :

Bash

```
mkdir -p resources/views/contacts
```

Puis vérifie :

Bash

```
ls resources/views
```

Tu devrais maintenant avoir :

```
contacts
```

3. Crée la vue `index.blade.php`

Exécute :

Bash

```
touch resources/views/contacts/index.blade.php
```

Puis ouvre le fichier :

Bash

```
nano resources/views/contacts/index.blade.php
```

Mets ceci :

blade

```
<!DOCTYPE html>
<html lang="fr">

<head>
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Annuaire de contacts</title>

    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>

<body class="min-h-screen bg-gray-100">

    <div class="mx-auto max-w-7xl px-6 py-10">

        <!-- En-tête -->
        <div class="mb-8 flex items-center justify-between">

            <div>
                <h1 class="text-3xl font-bold text-gray-900">
                    Annuaire de contacts
                </h1>

                <p class="mt-2 text-gray-500">
                    Gérez facilement vos contacts.
                </p>
            </div>

            <a
                href="{{ route('contacts.create') }}"
                class="rounded-lg bg-blue-600 px-5 py-3 font-semibold text-white transiti
            >
                + Ajouter un contact
            </a>

        </div>

        <!-- Message de succès -->
        @if(session('success'))

            <div class="mb-6 rounded-lg bg-green-100 px-5 py-4 text-green-700">
                {{ session('success') }}
            </div>

        @endif
```

```

<!-- Tableau -->
<div class="overflow-hidden rounded-xl bg-white shadow">

    <table class="w-full">

        <thead class="bg-gray-50">

            <tr>

                <th class="px-6 py-4 text-left text-sm font-semibold text-gray-700">
                    Prénom
                </th>

                <th class="px-6 py-4 text-left text-sm font-semibold text-gray-700">
                    Nom
                </th>

                <th class="px-6 py-4 text-left text-sm font-semibold text-gray-700">
                    Email
                </th>

                <th class="px-6 py-4 text-left text-sm font-semibold text-gray-700">
                    Téléphone
                </th>

                <th class="px-6 py-4 text-right text-sm font-semibold text-gray-700">
                    Actions
                </th>

            </tr>

        </thead>

        <tbody class="divide-y divide-gray-100">

            @forelse($contacts as $contact)

                <tr class="transition hover:bg-gray-50">

                    <td class="px-6 py-4">
                        {{ $contact->prenom }}
                    </td>

                    <td class="px-6 py-4 font-medium">
                        {{ $contact->nom }}
                    </td>

                    <td class="px-6 py-4 text-gray-600">
                        {{ $contact->email }}
                    </td>

```

```

<td class="px-6 py-4 text-gray-600">
    {{ $contact->telephone }}
</td>

<td class="px-6 py-4">

    <div class="flex justify-end gap-2">

        <!-- Consulter -->
        <a
            href="{{ route('contacts.show', $contact) }}"
            class="rounded-lg bg-gray-100 px-3 py-2 text-sm f
        >
            Voir
        </a>

        <!-- Modifier -->
        <a
            href="{{ route('contacts.edit', $contact) }}"
            class="rounded-lg bg-blue-50 px-3 py-2 text-sm fo
        >
            Modifier
        </a>

        <!-- Supprimer -->
        <form
            action="{{ route('contacts.destroy', $contact) }}"
            method="POST"
            onsubmit="return confirm('Voulez-vous vraiment su
        >

            @csrf

            @method('DELETE')

            <button
                type="submit"
                class="rounded-lg bg-red-50 px-3 py-2 text-sm
            >
                Supprimer
            </button>

        </form>

    </div>

</td>

</tr>

```

@empty

```

        <tr>

            <td
                colspan="5"
                class="px-6 py-12 text-center text-gray-500"
            >
                Aucun contact enregistré.
            </td>

        </tr>

    @endforelse

</tbody>

</table>

</div>

<!-- Pagination -->
<div class="mt-6">
    {{ $contacts->links() }}
</div>

</div>

</body>

</html>

```

Dans `nano` :

CTRL + O → Entrée → **CTRL + X**

4. Recharge la page

Va sur :

```
http://localhost:8000/contacts
```

Tu devrais maintenant avoir ton interface.

Comme ta base est probablement vide, tu verras :

Aucun contact enregistré.

avec le bouton :

+ Ajouter un contact

5. Mais attention : l'étape suivante

Quand tu cliqueras sur :

+ Ajouter un contact

Laravel cherchera cette vue :

```
resources/views/contacts/create.blade.php
```

Si elle n'existe pas, tu auras exactement le même type d'erreur :

```
View [contacts.create] not found.
```

Donc autant créer maintenant les **4 vues du CRUD** :

```
resources/views/  
|  
└─ contacts/  
    ├── index.blade.php  
    ├── create.blade.php  
    ├── show.blade.php  
    └─ edit.blade.php
```

Ensuite nous pourrons tester le CRUD dans l'ordre :

```
1 Ajouter  
  ↓  
2 Afficher la liste  
  ↓  
3 Consulter  
  ↓  
4 Modifier  
  ↓  
5 Supprimer
```

Ton erreur actuelle n'est donc pas une erreur de Laravel ou de base de données :
c'est simplement la vue `contacts/index.blade.php` qui manque.

```
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/laravel/annuaire-contact$ ls  
resources/views/  
create.blade.php edit.blade.php index.blade.php layouts  
show.blade.php welcome.blade.php  
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/laravel/annuaire-contact$
```

Ah, voilà l'origine exacte de l'erreur. ✓

Tes fichiers existent bien, mais **ils ne sont pas dans le dossier que ton contrôleur demande**.

Actuellement tu as :

```
resources/views/  
├─ create.blade.php  
├─ edit.blade.php  
├─ index.blade.php  
├─ show.blade.php  
├─ layouts/  
└─ welcome.blade.php
```

Alors que ton `ContactController` cherche probablement :

PHP

```
return view('contacts.index');
```

Laravel interprète cela comme :

```
resources/views/contacts/index.blade.php
```

Or ce fichier n'existe pas.

Solution recommandée

Comme ton projet est déjà organisé avec les fichiers directement dans `resources/views`, **ne recrée pas un dossier** `contacts`. Il suffit d'adapter le contrôleur.

Ouvre :

Bash

```
nano app/Http/Controllers/ContactController.php
```

Et vérifie les méthodes.

Elles doivent être comme ceci :

PHP

```
public function index()
{
    $contacts = Contact::latest()->paginate(10);

    return view('index', compact('contacts'));
}
```

Pour l'ajout :

PHP

```
public function create()
{
    return view('create');
}
```

Pour la consultation :

PHP

```
public function show(Contact $contact)
{
    return view('show', compact('contact'));
}
```

Pour la modification :

PHP

```
public function edit(Contact $contact)
{
    return view('edit', compact('contact'));
}
```

Donc les correspondances sont :

Contrôleur

Fichier réel

`view('index')`

`resources/views/index.blade.php`

`view('create')`

`resources/views/create.blade.php`

`view('show')`

`resources/views/show.blade.php`

`view('edit')`

`resources/views/edit.blade.php`

Vérifie ton `ContactController`

Tu peux aussi simplement exécuter :

Bash

```
cat app/Http/Controllers/ContactController.php
```

et envoie-moi le résultat ici.

Je pourrai vérifier ton contrôleur actuel et te dire exactement quelles lignes modifier, sans casser le reste de ton CRUD.

astuce: 'development'. La branche nouvellement créée peut être renommée avec :

astuce:

astuce: `git branch -m <nom>`

Dépôt Git vide initialisé dans

`/var/www/html/laravel/annuaire-contact/.git/`

`aminata@aminata-Lenovo-V15-`

`IGL:/var/www/html/laravel/annuaire-contact$ git remote`

`add origin aminatat443/annuaire-contact.git`

`git branch -M main`

`git push -u origin main`

error: le spécificateur de référence source main ne correspond à aucune référence

error: impossible de pousser des références vers '
aminatat443/annuaire-contact.git'

`aminata@aminata-Lenovo-V15-`

```
IGL:/var/www/html/laravel/annuaire-contact$ git remote
add origin aminatat443/annuaire-contact.git
error: la distante origin existe déjà.
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/laravel/annuaire-contact$ git branch -M
main
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/laravel/annuaire-contact$ git push -u
origin main
error: le spécificateur de référence source main ne
correspond à aucune référence
error: impossible de pousser des références vers '
aminatat443/annuaire-contact.git'
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/laravel/annuaire-contact$ git pull origin
main
fatal: impossible de trouver la référence distante main
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/laravel/annuaire-contact$ git branch
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/laravel/annuaire-contact$ git --fetch all
option inconnue : --fetch
usage: git [--version] [--help] [-C <path>] [-c <name>=
<value>]
        [--exec-path[=<path>]] [--html-path] [--man-path] [--
info-path]
        [-p | --paginate | -P | --no-pager] [--no-replace-
objects] [--bare]
        [--git-dir=<path>] [--work-tree=<path>] [--namespace=
<name>]
        [--super-prefix=<path>] [--config-env=<name>=
<envvar>]
        <command> [<args>]
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/laravel/annuaire-contact$ git fetch all
fatal: 'all' does not appear to be a git repository
fatal: Impossible de lire le dépôt distant.
```

Veillez vérifier que vous avez les droits d'accès
et que le dépôt existe.

```
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/laravel/annuaire-contact$ git fetch --all
Récupération de origin
```

```
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/laravel/annuaire-contact$ git branch  
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/laravel/annuaire-contact$ git pull origin  
main  
fatal: impossible de trouver la référence distante main  
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/laravel/annuaire-contact$
```

Oui, je vois exactement le problème. 👍

Ton dépôt Git local est vide en termes d'historique : tu n'as encore fait aucun commit.

C'est pour cela que :

```
git branch
```

ne retourne rien, et que :

```
git push -u origin main
```

donne :

```
error: le spécificateur de référence source main ne correspond à aucune référence
```

Pourquoi ?

Tu as bien fait :

Bash

```
git init  
git remote add origin ...  
git branch -M main
```

mais il manque l'étape essentielle :

```
git add → git commit → git push
```

Fais exactement ceci

Tu es déjà dans :

```
/var/www/html/laravel/annuaire-contact
```

1. Vérifie l'état

Bash

```
git status
```

Tu devrais voir tes fichiers Laravel comme fichiers non suivis.

2. Ajouter tous les fichiers

Bash

```
git add .
```

Puis vérifie :

Bash

```
git status
```

Tu devrais maintenant voir les fichiers en vert sous **Changes to be committed**.

3. Faire le premier commit

Bash

```
git commit -m "Initialisation du projet annuaire de contacts"
```

Tu devrais obtenir quelque chose ressemblant à :

```
[main (root-commit) xxxxxxx] Initialisation du projet annuaire de contacts
XX files changed
```

4. Vérifie ta branche

Bash

```
git branch
```

Cette fois tu dois avoir :

```
* main
```

5. Envoyer sur GitHub

Ton `origin` existe déjà, donc **ne refais surtout pas** :

Bash

```
git remote add origin ...
```

Fais simplement :

Bash

```
git push -u origin main
```

Ton problème résumé

Tu avais :

```
git init
  ↓
dépôt créé
  ↓
remote origin ajouté
  ↓
branche main renommée
  ↓
❌ aucun commit
  ↓
❌ rien à pousser
```

Il faut maintenant :

```
git add .
  ↓
git commit
  ↓
git push
  ↓
GitHub
```

⚠ **Et concernant tes commandes** `fetch`

Tu as essayé :

Bash

```
git --fetch all
```

✗ Incorrect.

Puis :

Bash

```
git fetch all
```

✗ Incorrect également : Git interprète `all` comme le nom d'un dépôt distant.

La bonne commande est :

Bash

```
git fetch --all
```

✓ Celle-là, tu l'as finalement exécutée correctement.

Mais ici, **ce n'était pas nécessaire**, car ton problème principal est l'absence de commit local.

Fais maintenant ces 4 commandes

Bash

```
git add .  
git commit -m "Initialisation du projet annuaire de contacts"  
git branch -M main  
git push -u origin main
```

Si `git push` te donne une nouvelle erreur, **copie-moi uniquement le résultat de ces commandes** et je te dirai exactement quoi faire.

commencons maintenant le developpement de khalil shop

Oui. On commence maintenant **KhalilShop**, mais cette fois comme un vrai projet e-commerce professionnel, pas comme un simple exercice CRUD.

Je vais te guider **étape par étape**, en construisant d'abord une base propre Laravel + Tailwind, puis le catalogue, panier, commandes, paiement, espace client et administration.

KHALILSHOP — Vision du projet

KhalilShop est une boutique **B2C** destinée aux particuliers.

Catégories principales

-  **Femme**
-  **Homme**
-  **Chaussures**
-  **Accessoires**
-  **Maison & Décoration**

Stack

Frontend

- |— Laravel Blade
- |— Tailwind CSS
- |— JavaScript
- |— Vite

Backend

- |— Laravel
- |— PHP
- |— Eloquent ORM
- |— API / logique métier

Base de données

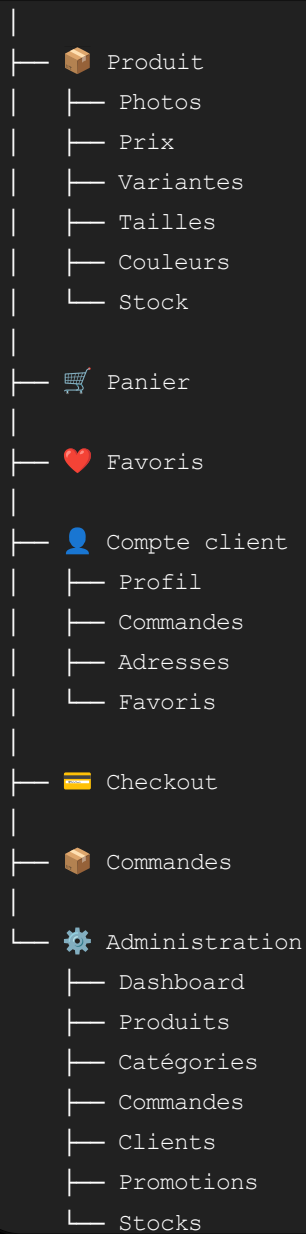
- |— PostgreSQL ou MySQL

1. Architecture que nous allons construire

Je recommande cette architecture :

KHALILSHOP

- |
- |—  Accueil
- |
- |—  Boutique
 - |— Femme
 - |— Homme
 - |— Chaussures
 - |— Accessoires
 - |— Maison & Décoration



2. Design UI/UX

On ne va pas faire un vieux site e-commerce avec :

- ✗ Bootstrap bleu/gris
- ✗ tableaux partout
- ✗ interface surchargée
- ✗ boutons énormes
- ✗ design daté

On vise :

- ✨ Premium
- ✨ Moderne
- ✨ Élégant
- ✨ Minimaliste
- ✨ Mobile-first

- ✨ Rapide
- ✨ Conversion-oriented

L'interface devra donner immédiatement l'impression d'une **marque de mode et lifestyle moderne**.

3. Première étape : créer le projet

Comme tu travailles déjà avec Laravel, commence par créer le projet dans ton dossier Laravel.

Bash

```
cd /var/www/html/laravel
```

Puis :

Bash

```
laravel new khalilshop
```

Ensuite :

Bash

```
cd khalilshop
```

Vérifie :

Bash

```
php artisan --version
```

Puis :

Bash

```
npm install
```

Et :

Bash

```
npm run build
```

4. Lancer Laravel

Terminal 1 :

Bash

```
php artisan serve
```

Tu devrais avoir :

```
http://127.0.0.1:8000
```

Terminal 2 :

Bash

```
npm run dev
```

Vite sera généralement disponible sur :

```
http://localhost:5173
```

⚠ **Mais pour consulter KhalilShop, tu utiliseras `localhost:8000`, pas `5173`.**

5. Première structure Laravel

Nous allons organiser le projet progressivement.

```
khalilshop/  
|  
├─ app/  
|   ├── Http/  
|   │   └─ Controllers/  
|   └─ Models/  
|   └─ ...  
├─ database/  
|   ├── migrations/  
|   └─ seeders/  
├─ resources/  
|   ├── css/  
|   ├── js/  
|   └─ views/
```

```
| layouts/
| components/
| home/
| products/
| cart/
| checkout/
| account/
| admin/
|
└─ routes/
    └─ web.php
```

6. Avant le catalogue : créer le Layout principal

C'est important.

Toutes les pages vont partager :

```
| TOPBAR
| Livraison • Promotions • Contact
|
| LOGO Recherche Compte ♥ Panier 🛒
|
| Femme Homme Chaussures Accessoires Maison
|
| CONTENU
|
| FOOTER
```

Cela permettra de garder une expérience cohérente sur tout le site.

7. Créer les dossiers

Dans le projet :

Bash

```
mkdir -p resources/views/layouts
mkdir -p resources/views/components
mkdir -p resources/views/home
mkdir -p resources/views/products
mkdir -p resources/views/cart
mkdir -p resources/views/checkout
```

```
mkdir -p resources/views/account
mkdir -p resources/views/admin
```

8. Créer le Layout

Bash

```
nano resources/views/layouts/app.blade.php
```

Pour commencer, mets :

blade

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>
        @yield('title', 'KhalilShop')
    </title>

    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>

<body class="bg-white text-gray-900 antialiased">

    {{-- Header --}}
    @include('components.header')

    {{-- Contenu --}}
    <main>
        @yield('content')
    </main>

    {{-- Footer --}}
    @include('components.footer')

</body>
</html>
```

9. Header KhalilShop

Crée :

Bash

```
nano resources/views/components/header.blade.php
```

On va commencer avec une première version propre :

blade

```
<header class="border-b border-gray-100 bg-white">

    {{-- Topbar --}}
    <div class="border-b border-gray-100 bg-gray-50">
        <div class="mx-auto flex max-w-7xl items-center justify-center px-4 py-2 text-xs">
            Livraison partout au Sénégal
        </div>
    </div>

    {{-- Navigation principale --}}
    <div class="mx-auto max-w-7xl px-4">

        <div class="flex h-20 items-center gap-8">

            {{-- Logo --}}
            <a
                href="{{ url('/') }}"
                class="shrink-0 text-2xl font-bold tracking-tight"
            >
                KHALIL<span class="font-light">SHOP</span>
            </a>

            {{-- Recherche --}}
            <div class="hidden flex-1 md:block">

                <div class="relative">

                    <input
                        type="search"
                        placeholder="Rechercher un produit..."
                        class="w-full rounded-full border border-gray-200 bg-gray-50 px-5"
                    >

                    <button
                        class="absolute right-2 top-1/2 -translate-y-1/2 rounded-full p-2"
                    >
                        
                    </button>
                </div>
            </div>
        </div>
    </div>
```



```
</div>
```

```
</div>
```

```
{{-- Actions --}}
```

```
<div class="ml-auto flex items-center gap-5">
```

```
<a
```

```
  href="#"
```

```
  class="hidden text-sm md:block"
```

```
>
```

```
  Compte
```

```
</a>
```

```
<a
```

```
  href="#"
```

```
  class="text-xl"
```

```
  aria-label="Favoris"
```

```
>
```

```
  ♥
```

```
</a>
```

```
<a
```

```
  href="#"
```

```
  class="relative text-xl"
```

```
  aria-label="Panier"
```

```
>
```



```
<span class="absolute -right-2 -top-2 flex h-5 w-5 items-center justify-
```

```
  center
```

```
</span>
```

```
</a>
```

```
</div>
```

```
</div>
```

```
{{-- Navigation catégories --}}
```

```
<nav class="hidden h-12 items-center justify-center gap-10 md:flex">
```

```
<a href="#" class="text-sm font-medium hover:text-gray-500">
```

```
  Femme
```

```
</a>
```

```
<a href="#" class="text-sm font-medium hover:text-gray-500">
```

```
  Homme
```

```
</a>
```

```
<a href="#" class="text-sm font-medium hover:text-gray-500">
```

```

        Chaussures
    </a>

    <a href="#" class="text-sm font-medium hover:text-gray-500">
        Accessoires
    </a>

    <a href="#" class="text-sm font-medium hover:text-gray-500">
        Maison & Décoration
    </a>

    <a href="#" class="text-sm font-medium text-red-600">
        Promotions
    </a>

</nav>

</div>

</header>

```

10. Footer

Crée :

Bash

```
nano resources/views/components/footer.blade.php
```

blade

```

<footer class="mt-20 border-t border-gray-100 bg-gray-950 text-white">

    <div class="mx-auto grid max-w-7xl gap-10 px-6 py-14 md:grid-cols-4">

        <div>
            <h2 class="text-xl font-bold">
                KHALIL<span class="font-light">SHOP</span>
            </h2>

            <p class="mt-4 text-sm leading-6 text-gray-400">
                Mode, accessoires et décoration pour créer
                un style qui vous ressemble.
            </p>
        </div>

        <div>
            <h3 class="font-semibold">

```

Boutique

</h3>

<ul class="mt-4 space-y-3 text-sm text-gray-400">

Femme

Homme

Chaussures

Accessoires

</div>

<div>

<h3 class="font-semibold">

Maison

</h3>

<ul class="mt-4 space-y-3 text-sm text-gray-400">

Décoration

Nouveautés

Promotions

</div>

<div>

<h3 class="font-semibold">

Service client

</h3>

<ul class="mt-4 space-y-3 text-sm text-gray-400">

Contact

Livraison

Retours

FAQ

</div>

</div>

<div class="border-t border-gray-800">

<div class="mx-auto max-w-7xl px-6 py-6 text-center text-sm text-gray-500">

© {{ date('Y') }} KhalilShop. Tous droits réservés.

```
</div>

</div>

</footer>
```

11. Créer la page d'accueil

Crée :

Bash

```
nano resources/views/home/index.blade.php
```

Pour le moment :

blade

```
@extends('layouts.app')

@section('title', 'KhalilShop – Mode & Maison')

@section('content')

<section class="mx-auto max-w-7xl px-4 py-20">

    <div class="grid items-center gap-12 md:grid-cols-2">

        <div>

            <p class="text-sm font-medium uppercase tracking-[0.25em] text-gray-500">
                Nouvelle collection
            </p>

            <h1 class="mt-5 text-5xl font-bold tracking-tight md:text-7xl">
                Votre style.
                <br>
                Votre univers.
            </h1>

            <p class="mt-6 max-w-lg text-lg leading-8 text-gray-600">
                Découvrez notre sélection de vêtements,
                chaussures, accessoires et décoration maison.
            </p>

            <div class="mt-8 flex gap-4">

                <a
```

```

        href="#"
        class="rounded-full bg-black px-7 py-4 text-sm font-semibold text-white"
    >
        Découvrir la boutique
    </a>

    <a
        href="#"
        class="rounded-full border border-gray-300 px-7 py-4 text-sm font-semi"
    >
        Voir les nouveautés
    </a>

</div>

</div>

<div class="flex min-h-[500px] items-center justify-center rounded-3xl bg-gray-100"
    >
        <span class="text-gray-400">
            Image Hero
        </span>

    </div>

</div>

</section>

@endsection

```

12. Route de la page d'accueil

Dans :

```
routes/web.php
```

mets :

PHP

```

<?php

use Illuminate\Support\Facades\Route;

Route::get('/', function () {

```

13. Tester

Lance :

```
Bash
```

```
php artisan serve
```

et ouvre :

```
http://localhost:8000
```

Tu dois maintenant avoir la première base visuelle de **KhalilShop**.



Ensuite, on passe au vrai cœur du projet

Je te propose de développer **dans cet ordre**, afin de ne pas construire quelque chose de désorganisé :

Phase 1 — UI/UX

- Header professionnel
- Mega-menu
- Hero
- Catégories
- Produits populaires
- Nouveautés
- Promotions
- Footer
- Responsive mobile

Phase 2 — Base de données

On créera notamment :

```
users  
categories  
products  
product_images  
product_variants  
sizes  
colors
```

```

carts
cart_items
wishlists
orders
order_items
addresses
coupons
reviews

```

Phase 3 — Catalogue

```

Catégories
  ↓
Produits
  ↓
Filtres
  ↓
Recherche
  ↓
Fiche produit

```

Phase 4 — Achat

```

Produit
  ↓
Panier
  ↓
Adresse
  ↓
Livraison
  ↓
Paiement
  ↓
Commande

```

Phase 5 — Administration

```

Dashboard
├─ Produits
├─ Catégories
├─ Stocks
├─ Commandes
├─ Clients
├─ Promotions
└─ Statistiques

```

On va donc commencer par la fondation : Laravel + Tailwind + Layout + Header + Footer + Home. Puis nous construirons la base de données du catalogue avant de

développer les pages produits.

```
avant de continuer j ai une erreur
Illuminate\Database\QueryException
vendor/laravel/framework/src/Illuminate/Database/Connect
ion.php:838

SQLSTATE[42S02]: Base table or view not found: 1146
Table 'khalilshop.sessions' doesn't exist (Connection:
mysql, Host: 127.0.0.1, Port: 3306, Database: khalilshop,
SQL: select * from `sessions` where `id` =
ZWMX0cezBGEQJvVsCBvbt3lvDD2uHhYpCPA2Sdxs
limit 1)

LARAVEL12.68.0

PHP8.2.33

svgUNHANDLED

CODE 42S02

svg500

svgGET

http://localhost:8000
```

Oui. Cette erreur est très claire : **Laravel est configuré pour stocker les sessions dans MySQL, mais la table `sessions` n'existe pas dans ta base `khalilshop`.**

Tu utilises :

```
Laravel 12.68.0
PHP 8.2.33
MySQL 127.0.0.1:3306
Base : khalilshop
```

L'erreur vient de :

```
SQLSTATE[42S02]
Table 'khalilshop.sessions' doesn't exist
```

 **Solution recommandée**

Laravel 12 utilise souvent le driver de session `database`. Il faut donc créer la table `sessions`.

Dans ton terminal, place-toi dans le projet :

Bash

```
cd /var/www/html/laravel/khalilshop
```

Puis lance :

Bash

```
php artisan migrate
```

Si tout se passe bien

Tu devrais voir quelque chose comme :

```
INFO  Running migrations.
```

```
2026_..._create_cache_table ..... DONE
```

```
2026_..._create_jobs_table ..... DONE
```

```
2026_..._create_sessions_table ..... DONE
```

Ensuite recharge :

```
http://localhost:8000
```



Si `php artisan migrate` te dit qu'il n'y a aucune migration pour `sessions`

Vérifie :

Bash

```
ls database/migrations
```

Tu devrais avoir un fichier ressemblant à :

```
xxxx_xx_xx_XXXXXX_create_sessions_table.php
```

Si tu ne l'as pas, exécute :

Bash

```
php artisan make:session-table
```

Puis :

Bash

```
php artisan migrate
```

⚠ Une chose importante avant de continuer KhalilShop

Comme tu viens de créer le projet, **ne crée pas encore les tables** `products`, `categories`, `orders`, **etc.**

On doit d'abord vérifier que la base Laravel fonctionne correctement.

Fais simplement :

Bash

```
cd /var/www/html/laravel/khalilshop
```

```
php artisan migrate
```

Puis :

Bash

```
php artisan migrate:status
```

Et envoie-moi le résultat.

Je vérifierai que **MySQL + Laravel + migrations + sessions** sont correctement configurés avant qu'on commence la conception de la base de données de KhalilShop.

```
0001_01_01_000000_create_users_table
..... 327.70ms DONE
0001_01_01_000001_create_cache_table
..... 193.16ms DONE
0001_01_01_000002_create_jobs_table
```

```
..... 241.99ms DONE
```

```
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/khalilshop$ php artisan make:session-  
table
```

```
ERROR Migration already exists.
```

```
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/khalilshop$
```

Oui, c'est normal. 👍

Tes **3 migrations Laravel par défaut** ont bien été exécutées :

```
create_users_table  DONE  
create_cache_table  DONE  
create_jobs_table   DONE
```

Mais la migration `sessions` **existe déjà dans ton projet**, seulement elle n'a pas encore été exécutée. C'est pour cela que :

Bash

```
php artisan make:session-table
```

répond :

```
ERROR Migration already exists.
```

Fais ceci maintenant

Dans ton projet :

Bash

```
cd /var/www/html/khalilshop
```

Vérifie que la migration existe :

Bash

```
ls database/migrations | grep session
```

Tu devrais voir quelque chose comme :

```
xxxx_xx_xx_XXXXXX_create_sessions_table.php
```

Ensuite lance simplement :

Bash

```
php artisan migrate
```

Laravel devrait exécuter uniquement la migration `sessions` restante.

Tu devrais obtenir :

```
xxxx_xx_xx_XXXXXX_create_sessions_table ..... DONE
```

Ensuite vérifie

Bash

```
php artisan migrate:status
```

Tu devrais avoir les migrations avec :

```
Ran
```

notamment :

```
create_users_table      Ran
create_cache_table      Ran
create_jobs_table       Ran
create_sessions_table    Ran
```

Puis recharge :

```
http://localhost:8000
```

 **Ne fais surtout pas**

Bash

```
php artisan migrate:fresh
```

pour l'instant, car cette commande supprimerait toutes les tables existantes de ta base

```
khalilshop
```

Fais maintenant uniquement :

Bash

```
ls database/migrations | grep session  
php artisan migrate  
php artisan migrate:status
```

Envoie-moi le résultat de ces trois commandes et on vérifie ensemble que la base de KhalilShop est propre avant de continuer.

```
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/khalilshop$ ls database/migrations |  
grep session  
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/khalilshop$
```

Exactement : **la commande ne retourne rien**, donc il n'y a actuellement **aucune migration sessions dans database/migrations**.

C'est pour cela que Laravel cherche :

```
khalilshop.sessions
```

mais que la table n'existe pas.

1. Crée la migration sessions

Dans ton projet :

Bash

```
php artisan make:session-table
```

Tu avais obtenu précédemment :

```
ERROR Migration already exists.
```

Cela peut arriver si Laravel considère qu'une migration de session existe déjà ailleurs ou qu'il y a un état incohérent. Commençons par vérifier toutes les migrations :

Bash

```
ls -la database/migrations
```

Puis :

Bash

```
grep -R "create_sessions_table" database/migrations
```

Et également :

Bash

```
grep -n "SESSION_DRIVER" .env
```

2. Solution rapide si `SESSION_DRIVER=database`

Si ton `.env` contient :

env

```
SESSION_DRIVER=database
```

tu peux temporairement utiliser les sessions fichiers, ce qui évite d'avoir besoin d'une table MySQL :

env

```
SESSION_DRIVER=file
```

Puis nettoie le cache de configuration :

Bash

```
php artisan config:clear
```

Ensuite :

Bash

```
php artisan serve
```

Recharge :

```
http://localhost:8000
```

Pour KhalilShop, je te recommande toutefois de garder cette configuration temporaire uniquement pour débloquer le projet. Plus tard, nous déciderons ensemble si les sessions doivent rester en fichier ou passer en base de données selon l'architecture d'authentification.

Mais avant de modifier quoi que ce soit

Exécute ces **3 commandes** et envoie-moi le résultat :

Bash

```
ls -la database/migrations
```

Bash

```
grep -R "create_sessions_table" database/migrations
```

Bash

```
grep "SESSION_DRIVER" .env
```

Avec ça, je pourrai te dire précisément pourquoi Laravel demande `sessions` alors qu'aucune migration de session n'apparaît.

```
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/khalilshop$ ls database/migrations |
grep session
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/khalilshop$ ls -la database/migrations
total 20
drwxrwxr-x 2 aminata aminata 4096 mar 14 16:54 .
drwxrwxr-x 5 aminata aminata 4096 uut 27 21:10 ..
-rw-rw-r-- 1 aminata aminata 1473 mar 14 16:54
0001_01_01_000000_create_users_table.php
-rw-rw-r-- 1 aminata aminata 867 mar 14 16:54
0001_01_01_000001_create_cache_table.php
-rw-rw-r-- 1 aminata aminata 1812 mar 14 16:54
0001_01_01_000002_create_jobs_table.php
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/khalilshop$ grep -R
```

```
"create_sessions_table" database/migrations
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/khalilshop$ grep -n
"SESSION_DRIVER" .env
30:SESSION_DRIVER=database
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/khalilshop$ php artisan config:clear
```

INFO Configuration cache cleared successfully.

```
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/khalilshop$ php artisan config:clear
```

INFO Configuration cache cleared successfully.

```
aminata@aminata-Lenovo-V15-
IGL:/var/www/html/khalilshop$
```

Parfait, maintenant on a identifié **exactement** le problème. ✓

Ton projet contient seulement :

```
0001_01_01_000000_create_users_table.php
0001_01_01_000001_create_cache_table.php
0001_01_01_000002_create_jobs_table.php
```

et ton `.env` contient :

env

```
SESSION_DRIVER=database
```

Donc Laravel dit :

« Je dois enregistrer les sessions dans MySQL »

mais ta base n'a pas de table `sessions`.

La meilleure solution maintenant

Puisque nous sommes encore au début de **KhalilShop**, je te recommande de créer correctement la migration `sessions`.

Exécute :

Bash

```
php artisan make:migration create_sessions_table
```

Puis ouvre le fichier qui vient d'être créé :

Bash

```
ls database/migrations
```

Tu verras un nouveau fichier du genre :

```
2026_08_28_XXXXXX_create_sessions_table.php
```

Ouvre-le :

Bash

```
nano database/migrations/2026_08_28_XXXXXX_create_sessions_table.php
```

⚠ Remplace `2026_08_28_XXXXXX` par le nom exact généré chez toi.

Mets :

PHP

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->text('payload');
            $table->integer('last_activity')->index();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('sessions');
    }
}
```

```
}  
};
```

Enregistre :

```
CTRL + O  
Entrée  
CTRL + X
```

Puis lance :

Bash

```
php artisan migrate
```

Tu devrais maintenant avoir :

```
create_sessions_table ..... DONE
```

Vérifie

Bash

```
php artisan migrate:status
```

Tu dois avoir :

```
create_users_table      Ran  
create_cache_table      Ran  
create_jobs_table       Ran  
create_sessions_table   Ran
```

Puis :

Bash

```
php artisan serve
```

et ouvre :

```
http://localhost:8000
```

Pourquoi je préfère cette solution pour KhalilShop

On pourrait simplement changer :

```
env
```

```
SESSION_DRIVER=database
```

en :

```
env
```

```
SESSION_DRIVER=file
```

mais Laravel 12 a été configuré chez toi pour utiliser les sessions en base. **Autant corriger proprement la structure de la base dès maintenant**, avant de commencer l'authentification client, le panier et les commandes.

Une fois cette erreur réglée, **on ne touche plus à la configuration de base au hasard** : on va construire KhalilShop proprement, migration par migration.

```
aminata@aminata-Lenovo-V15-
```

```
IGL:/var/www/html/khalilshop$ php artisan make:migration  
create_sessions_table
```

```
INFO Migration
```

```
[database/migrations/2026_08_28_122207_create_sessions_table.php] created successfully.
```

```
aminata@aminata-Lenovo-V15-
```

```
IGL:/var/www/html/khalilshop$ php artisan migrate
```

```
INFO Running migrations.
```

```
2026_08_28_122207_create_sessions_table
```

```
..... 11.17ms FAIL
```

```
Illuminate\Database\QueryException
```

```
SQLSTATE[42S01]: Base table or view already exists:  
1050 Table 'sessions' already exists (Connection: mysql,  
Host: 127.0.0.1, Port: 3306, Database: khalilshop, SQL:  
create table sessions (id varchar(255) notnull, user_id
```

```
bigint unsigned null, ip_address varchar(45) null,  
user_agent text null, payload text not null,  
last_activity int not null, primary key (id)) default  
character set utf8mb4 collate 'utf8mb4_unicode_ci')
```

```
at  
vendor/laravel/framework/src/Illuminate/Database/Connect  
ion.php:838
```

```
834 |         $exceptionType = $this->  
>isUniqueConstraintError($e)  
835 |         ?  
UniqueConstraintViolationException::class  
836 |         : QueryException::class;  
837 |  
→ 838 |         throw new $exceptionType(  
839 |             $this->getNameWithReadWriteType(),  
840 |             $query,  
841 |             $this->prepareBindings($bindings),  
842 |             $e,
```

```
+9 vendor frames
```

```
10  
database/migrations/2026_08_28_122207_create_sessio  
ns_table.php:11
```

```
Illuminate\Support\Facades\Facade::__callStatic()  
+26 vendor frames
```

```
37 artisan:16
```

```
Illuminate\Foundation\Application::handleCommand()
```

```
aminata@aminata-Lenovo-V15-  
IGL:/var/www/html/khalilshop$
```

Parfait, cette fois l'erreur nous donne l'information décisive. 👍

Ce qui s'est passé

La table `sessions` existe déjà dans MySQL :

```
SQLSTATE[42S01]: Base table or view already exists:  
1050 Table 'sessions' already exists
```

Mais Laravel ne la connaît pas dans sa table de suivi des migrations (`migrations`).

Donc nous avons actuellement :

```
MySQL
├─ sessions ✔ existe
|
Laravel migrations
├─ create_sessions_table ✗ pas encore marqué comme exécuté
```

Quand tu fais :

Bash

```
php artisan migrate
```

Laravel essaie donc de créer une deuxième fois `sessions` → erreur.

✔ Solution propre

Ne supprime pas la table `sessions`.

Puisque la table existe déjà, nous allons simplement faire en sorte que Laravel sache qu'elle existe.

1. Vérifie d'abord les migrations

Lance :

Bash

```
php artisan migrate:status
```

Envoie-moi le résultat.

Mais surtout, **ne relance pas** `php artisan migrate` **pour le moment**, car il va continuer à essayer de créer `sessions`.

2. Vérifie la structure de la table

Tu peux également exécuter :

Bash

```
php artisan tinker
```

Puis :

PHP

```
Schema::hasTable('sessions');
```

Tu devrais obtenir :

```
=> true
```

Puis quitte Tinker :

```
exit
```

3. Pourquoi `make:session-table` avait dit "Migration already exists"

C'est maintenant cohérent : Laravel possède probablement une définition de migration/session quelque part dans la configuration ou dans le squelette du projet, tandis que **le fichier de migration n'était simplement pas présent dans**

```
database/migrations.
```

Et surtout, ta base MySQL avait déjà la table `sessions`.



Ne fais pas `migrate:fresh`

Je ne veux pas que tu fasses :

Bash

```
php artisan migrate:fresh
```

car cette commande va supprimer les tables de ta base `khalilshop`.

Pour l'instant, nous voulons simplement **synchroniser Laravel avec la base existante**.

Fais maintenant uniquement :

Bash

```
php artisan migrate:status
```

Envoie-moi le résultat complet. À partir de ça, je te donnerai la commande exacte pour remettre les migrations de KhalilShop dans un état propre, sans supprimer ta base.

